PTI_AngelOmarGarciaHernandez_2025_CBI_Ana

August 7, 2025

[7]:
```
# ==============================================================================
# CELDA 1: CONFIGURACIÓN INICIAL Y ESTILO GRÁFICO PARA LA TESIS
# ==============================================================================

# ------------------------------------------------------------------------------
# 1. IMPORTACIÓN DE LIBRERÍAS
# ------------------------------------------------------------------------------
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.patheffects as path_effects
from scipy.integrate import solve_ivp
import os

# ------------------------------------------------------------------------------
# 2. DEFINICIÓN DE LA IDENTIDAD GRÁFICA (PALETA DE COLORES)
# ------------------------------------------------------------------------------
# Paleta diseñada para alta legibilidad, contraste y estética profesional.
# Accesible para la mayoría de los tipos de daltonismo.
PALETA = {
    # Colores primarios para datos vs. modelo
    'datos': '#0072B2',         # Azul académico
    'modelo': '#D55E00',        # Naranja quemado

    # Secuencia para las 6 olas epidémicas
    'ola_1': '#2a9d8f',
    'ola_2': '#e9c46a',
    'ola_3': '#f4a261',
    'ola_4': '#e76f51',
    'ola_5': '#d62828',
    'ola_6': '#8e9aaf',

    # Colores para elementos de soporte
    'texto_principal': '#333333',
    'texto_secundario': '#555555',
    'rejilla': '#DDDDDD',
```

```
    'fondo': '#FFFFFF',
    'anotacion_primaria': '#d62728', # Rojo para alertas importantes
    'anotacion_secundaria': '#7f7f7f' # Gris para líneas contextuales
}

# ------------------------------------------------------------------------------
# 3. CONFIGURACIÓN GLOBAL DE MATPLOTLIB (ESTILO DE PUBLICACIÓN)
# ------------------------------------------------------------------------------
# Este bloque asegura que TODAS las figuras sigan el mismo estilo.
plt.rcParams.update({
    # Tipografía (LaTeX para consistencia matemática)
    'font.family': 'serif',
    'font.serif': ['Computer Modern Roman', 'Times New Roman'],
    'text.usetex': True,
    'text.latex.preamble': r'\usepackage{amsmath}',

    # Jerarquía de tamaños de fuente
    'figure.titlesize': 18,
    'axes.titlesize': 14,
    'axes.labelsize': 12,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'legend.fontsize': 10,

    # Estilo de líneas y ejes
    'axes.linewidth': 1.0,
    'lines.linewidth': 2.2,
    'lines.markersize': 8,

    # Colores y rejilla
    'figure.facecolor': PALETA['fondo'],
    'axes.facecolor': PALETA['fondo'],
    'axes.labelcolor': PALETA['texto_principal'],
    'xtick.color': PALETA['texto_principal'],
    'ytick.color': PALETA['texto_principal'],
    'grid.color': PALETA['rejilla'],
    'grid.linestyle': '--',

    # Eliminación de "ruido" visual (ejes superior y derecho)
    'axes.spines.top': False,
    'axes.spines.right': False,

    # Configuración de guardado de figuras
    'figure.dpi': 300,
    'savefig.dpi': 300,
    'savefig.bbox': 'tight',
    'savefig.pad_inches': 0.1
```

```
})

# ------------------------------------------------------------------------------
# 4. PARÁMETROS CANÓNICOS DEL MODELO (Basados en Monte Carlo Híbrido)
# ------------------------------------------------------------------------------
# Estos son los valores definitivos que se usarán en todas las simulaciones.
PARAMETROS_OLAS = {
    'Ola 1 (Original)': {'R0': 1.92, 'D': 10, 's0': 1.000, 'color':␣
 ↪PALETA['ola_1']},
    'Ola 2 (Alpha)':    {'R0': 1.65, 'D': 11, 's0': 0.740, 'color':␣
 ↪PALETA['ola_2']},
    'Ola 3 (Delta)':    {'R0': 2.84, 'D': 9,  's0': 0.471, 'color':␣
 ↪PALETA['ola_3']},
    'Ola 4 (Ómicron)':  {'R0': 1.82, 'D': 7,  's0': 0.701, 'color':␣
 ↪PALETA['ola_4']},
    'Ola 5 (Ómicron Subs.)': {'R0': 5.41, 'D': 6,  's0': 0.416, 'color':␣
 ↪PALETA['ola_5']},
    'Ola 6':            {'R0': 4.82, 'D': 6,  's0': 0.315, 'color':␣
 ↪PALETA['ola_6']}
}

# Calculamos gamma (tasa de recuperación) para cada ola
for ola in PARAMETROS_OLAS:
    PARAMETROS_OLAS[ola]['gamma'] = 1.0 / PARAMETROS_OLAS[ola]['D']

# ------------------------------------------------------------------------------
# 5. EVENTOS CLAVE DE LA PANDEMIA EN MÉXICO
# ------------------------------------------------------------------------------
# Se usarán para anotar las gráficas de series temporales.
EVENTOS_CLAVE = {
    'Primer Caso': '2020-02-27',
    'OMS declara pandemia': '2020-03-11',
    'Inicio Sana Distancia': '2020-03-23',
    'Declaración Emergencia Sanitaria': '2020-03-30',
    'Inicio "Nueva Normalidad"': '2020-06-01',
    'Llegan primeras vacunas': '2020-12-23',
    'Inicio Vacunación': '2020-12-24',
    'Detectan Variante Delta': '2021-05-02',
    'Fin de Emergencia Sanitaria': '2023-05-09'
}

# ------------------------------------------------------------------------------
# 6. CREACIÓN DE CARPETA PARA GUARDAR FIGURAS
# ------------------------------------------------------------------------------
if not os.path.exists('Figuras Tesis'):
    os.makedirs('Figuras Tesis')
```

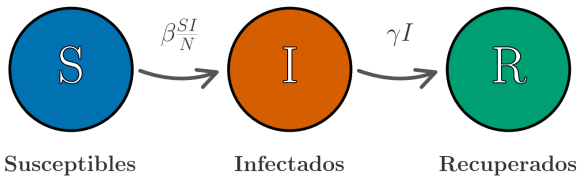
```
print("  Entorno de visualización y parámetros configurados con éxito.")
```

```
 Entorno de visualización y parámetros configurados con éxito.
```

[8]:
```
# ============================================================================
# CELDA 2: FIGURA 2.1 - DIAGRAMA DE FLUJO DEL MODELO SIR
# ============================================================================
import matplotlib.patches as mpatches
import matplotlib.patheffects as path_effects

# --- Configuración de la figura ---
fig_2_1, ax = plt.subplots(figsize=(10, 6))
ax.set_aspect('equal')
ax.set_xlim(0, 10)
ax.set_ylim(0, 7)
ax.axis('off')

# --- Estilos para los elementos del diagrama ---
comp_style = dict(radius=0.7, ec='black', lw=1.5, zorder=2)
text_style = dict(ha='center', va='center', fontsize=28, color='white',␣
 ↪zorder=3,
                  path_effects=[path_effects.withStroke(linewidth=1,␣
 ↪foreground='black')])
label_style = dict(ha='center', va='center', fontsize=12, zorder=3,␣
 ↪color=PALETA['texto_principal'])
arrow_style = dict(arrowstyle='->', mutation_scale=25, lw=2.0,␣
 ↪connectionstyle="arc3,rad=0.2", zorder=1)
rate_text_style = dict(ha='center', va='center', fontsize=14, zorder=3)
halo_effect = [path_effects.withStroke(linewidth=3, foreground=PALETA['fondo'])]

# --- Posiciones y etiquetas ---
posiciones = {'S': (2.5, 3.5), 'I': (5, 3.5), 'R': (7.5, 3.5)}
etiquetas = {'S': r'\textbf{Susceptibles}', 'I': r'\textbf{Infectados}', 'R':␣
 ↪r'\textbf{Recuperados}'}
colores_comp = {'S': PALETA['datos'], 'I': PALETA['modelo'], 'R': '#009E73'} #␣
 ↪Verde para Recuperados

# --- Dibujar compartimentos ---
for comp, pos in posiciones.items():
    ax.add_patch(mpatches.Circle(pos, fc=colores_comp[comp], **comp_style))
    ax.text(pos[0], pos[1], comp, **text_style)
    ax.text(pos[0], pos[1] - 1.1, etiquetas[comp], **label_style)

# --- Dibujar flujos y tasas ---
# Infección (S -> I)
```

```
ax.add_patch(mpatches.FancyArrowPatch((3.25, 3.5), (4.25, 3.5),␣
 ↪color=PALETA['texto_secundario'], **arrow_style))
ax.text(3.75, 3.9, r'$\beta \frac{S I}{N}$', color=PALETA['texto_principal'],␣
 ↪path_effects=halo_effect, **rate_text_style)
# Recuperación (I -> R)
ax.add_patch(mpatches.FancyArrowPatch((5.75, 3.5), (6.75, 3.5),␣
 ↪color=PALETA['texto_secundario'], **arrow_style))
ax.text(6.25, 3.9, r'$\gamma I$', color=PALETA['texto_principal'],␣
 ↪path_effects=halo_effect, **rate_text_style)

# --- Título y guardado ---
fig_2_1.suptitle(r'\textbf{Figura 2.1: Diagrama de Flujo del Modelo de␣
 ↪Compartimentos SIR}', y=0.92)
fig_2_1.savefig('Figuras Tesis/Figura_2_1_Diagrama_SIR.pdf')
fig_2_1.savefig('Figuras Tesis/Figura_2_1_Diagrama_SIR.png')
plt.show()

print(" Figura 2.1 generada y guardada.")
```

Figura 2.1: Diagrama de Flujo del Modelo de Compartimentos SIR




```
 Figura 2.1 generada y guardada.
```

[ ]:
```
# ============================================================================
# CELDA 3: FIGURA 2.2 - ESTABILIDAD Y BIFURCACIÓN TRANSCRÍTICA
# ============================================================================
```

```
# --- Configuración de la figura de 2 paneles ---
fig_2_2, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
fig_2_2.suptitle(r'\textbf{Figura 2.2: Análisis de Estabilidad y Bifurcación␣
 ↪del Modelo SIR}', y=1.02)

# --- Panel A: Estabilidad Lineal ---
ax1.set_title(r'\textbf{A)} Estabilidad del Equilibrio Libre de Enfermedad')
s_vals = np.linspace(0, 1, 200)
gamma_ejemplo = 1/10  # Tasa de recuperación de ejemplo (10 días)
R0_vals = [0.5, 1.0, 1.5, 2.5]
colores_estabilidad = plt.get_cmap('viridis_r', len(R0_vals) + 1)

ax1.axhline(0, color=PALETA['texto_secundario'], linestyle='--', lw=1.5)
for i, R0 in enumerate(R0_vals):
    autovalor = gamma_ejemplo * (R0 * s_vals - 1)
    ax1.plot(s_vals, autovalor, color=colores_estabilidad(i), label=f'$R_0 =␣
 ↪{R0:.1f}$')

ax1.set_xlabel(r'Fracción de Susceptibles ($S/N$)')
ax1.set_ylabel(r'Autovalor Dominante ($\lambda_2$)')
ax1.set_xlim(0, 1)
ax1.legend(title=r'Valor de $R_0$')
ax1.grid(True)
ax1.text(0.5, 0.04, 'Crecimiento Epidémico\n($\\lambda_2 > 0$)', ha='center',␣
 ↪va='bottom', fontsize=9, bbox=dict(facecolor='white', alpha=0.5,␣
 ↪edgecolor='none'))
ax1.text(0.5, -0.04, 'Decaimiento\n($\\lambda_2 < 0$)', ha='center', va='top',␣
 ↪fontsize=9, bbox=dict(facecolor='white', alpha=0.5, edgecolor='none'))

# --- Panel B: Bifurcación Transcrítica ---
ax2.set_title(r'\textbf{B)} Bifurcación Transcrítica en $R_0=1$')
R0_bifurcacion = np.linspace(0, 3, 300)
# Rama estable del Equilibrio Libre de Enfermedad (I=0)
ax2.plot(R0_bifurcacion[R0_bifurcacion <= 1], np.
 ↪zeros_like(R0_bifurcacion[R0_bifurcacion <= 1]),
         color=PALETA['datos'], lw=3, label='ELE Estable')
# Rama inestable del Equilibrio Libre de Enfermedad (I=0)
ax2.plot(R0_bifurcacion[R0_bifurcacion > 1], np.
 ↪zeros_like(R0_bifurcacion[R0_bifurcacion > 1]),
         color=PALETA['modelo'], linestyle='--', lw=2, label='ELE Inestable')
# Rama del Equilibrio Endémico (I > 0) que emerge en R0=1
i_endemico = 1 - 1/R0_bifurcacion[R0_bifurcacion > 1]
ax2.plot(R0_bifurcacion[R0_bifurcacion > 1], i_endemico * 0.3, # Escalado para␣
 ↪visualización
         color=PALETA['datos'], lw=3, label='Eq. Endémico Estable')
```

```
ax2.axvline(1.0, color=PALETA['texto_secundario'], linestyle=':', lw=1.5,␣
 ↪label=r'Umbral Crítico $R_0=1$')
ax2.set_xlabel(r'Número Reproductivo Básico ($R_0$)')
ax2.set_ylabel(r'Fracción de Infectados en Equilibrio ($I^*/N$)')
ax2.set_yticks([])
ax2.set_ylim(-0.05, 0.3)
ax2.grid(True)
ax2.legend()

# --- Guardado de la figura compuesta ---
plt.tight_layout(rect=[0, 0, 1, 0.96])
fig_2_2.savefig('Figuras Tesis/Figura_2_2_Estabilidad_Bifurcacion.pdf')
fig_2_2.savefig('Figuras Tesis/Figura_2_2_Estabilidad_Bifurcacion.png')
plt.show()

print("  Figura 2.2 generada y guardada.")
```

[ ]:
```
# ==============================================================================
# CELDA FINAL Y CORREGIDA: FIGURA 3.1 - AJUSTES FINALES DE TAMAÑO Y LEYENDAS
# ==============================================================================
# Este bloque de código es autocontenido y genera la versión definitiva de la␣
 ↪Figura 3.1
# con los ajustes finales de legibilidad.
# ==============================================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from matplotlib.patches import Patch
from scipy.signal import find_peaks
import os
import locale

# --- Configuración del locale a español para meses ---
try:
    locale.setlocale(locale.LC_TIME, 'es_ES.UTF-8')
except locale.Error:
    try:
        locale.setlocale(locale.LC_TIME, 'Spanish_Spain')
    except locale.Error:
        print("Locale en español no encontrado. Los meses pueden aparecer en␣
 ↪inglés.")

# ------------------------------------------------------------------------------
# 1. CONFIGURACIÓN (PALETA, PARÁMETROS, ESTILO)
# ------------------------------------------------------------------------------
PALETA = {
```

```
    'datos': '#0072B2', 'modelo': '#D55E00', 'ola_1': '#2a9d8f', 'ola_2':␣
 ↪'#e9c46a',
    'ola_3': '#f4a261', 'ola_4': '#e76f51', 'ola_5': '#d62828',
    'texto_principal': '#333333', 'rejilla': '#DDDDDD', 'fondo': '#FFFFFF',
    'pico': '#d62728', 'valle': '#1f77b4', 'anotacion_politica': '#404040'
}
# --- AJUSTE: Se reintroducen niveles manuales para control total de la␣
 ↪posición ---
EVENTOS_CLAVE = [
    {'texto': 'Primer Caso', 'fecha': '2020-02-27', 'nivel': 0},
    {'texto': 'Emergencia Sanitaria', 'fecha': '2020-03-30', 'nivel': 1},
    {'texto': 'Inicio "Nueva Normalidad"', 'fecha': '2020-06-01', 'nivel': 0},
    {'texto': 'Inicio Vacunación', 'fecha': '2020-12-24', 'nivel': 1}, # Nivel␣
 ↪alto
    {'texto': 'Llega Variante Delta', 'fecha': '2021-05-02', 'nivel': 0}, #␣
 ↪Nivel bajo para evitar colisión
    {'texto': 'Llega Variante Ómicron', 'fecha': '2021-12-03', 'nivel': 0},
    {'texto': 'Fin Emergencia Sanitaria', 'fecha': '2023-05-09', 'nivel': 0}
]
# --- AJUSTE: Se aumentan los tamaños de fuente ---
plt.rcParams.update({
    'font.family': 'serif', 'font.serif': ['Computer Modern Roman'], 'text.
 ↪usetex': True,
    'text.latex.preamble': r'\usepackage{amsmath}', 'figure.titlesize': 22,
    'axes.titlesize': 16, 'axes.labelsize': 14, 'xtick.labelsize': 12,
    'ytick.labelsize': 12, 'legend.fontsize': 12, 'axes.linewidth': 1.0,
    'lines.linewidth': 2.5, 'figure.facecolor': PALETA['fondo'],
    'axes.facecolor': PALETA['fondo'], 'axes.labelcolor':␣
 ↪PALETA['texto_principal'],
    'xtick.color': PALETA['texto_principal'], 'ytick.color':␣
 ↪PALETA['texto_principal'],
    'grid.color': PALETA['rejilla'], 'grid.linestyle': '--', 'axes.spines.top':␣
 ↪False,
    'axes.spines.right': False, 'figure.dpi': 300, 'savefig.dpi': 300, 'savefig.
 ↪bbox': 'tight'
})
if not os.path.exists('Figuras Tesis'):
    os.makedirs('Figuras Tesis')

# ------------------------------------------------------------------------
# 2. CARGA Y PROCESAMIENTO DE DATOS
# ------------------------------------------------------------------------
nombre_archivo = 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv'
print(f"Cargando datos desde '{nombre_archivo}'...")
df_raw = pd.read_csv(nombre_archivo)
df_nacional = df_raw[df_raw['nombre'] == 'Nacional'].copy()
```

```
casos_diarios_raw = df_nacional.iloc[:, 3:].astype(float).T
datos_df = pd.DataFrame(index=pd.to_datetime(casos_diarios_raw.index,␣
 ↪format='%d-%m-%Y'))
datos_df['casos_suavizados'] = datos_df.assign(casos_crudos=casos_diarios_raw.
 ↪values)['casos_crudos'].rolling(window=7, center=True, min_periods=1).mean()
print("Datos cargados y procesados.")

# ------------------------------------------------------------------------
# 3. DETECCIÓN DE HITOS DINÁMICOS Y OLAS (DEFINICIÓN DE 5 OLAS)
# ------------------------------------------------------------------------
print("Definiendo 5 olas con el método valle a valle optimizado...")
indices_valles, _ = find_peaks(-datos_df['casos_suavizados'], distance=100,␣
 ↪prominence=2500)
fronteras = np.unique(np.concatenate(([0], indices_valles, [len(datos_df)-1])))
indices_picos, _ = find_peaks(datos_df['casos_suavizados'], distance=100,␣
 ↪prominence=3000)

olas_con_hitos = []
nombres_olas_base = [
    'Ola 1 (Original)', 'Ola 2 (Alpha/Gamma)', 'Ola 3 (Delta)',
    'Ola 4 (Ómicron BA.1)', 'Ola 5 (Ómicron Subs.)'
]
for i in range(len(fronteras) - 1):
    if i < len(nombres_olas_base):
        inicio_idx, fin_idx = fronteras[i], fronteras[i+1]
        pico_idx_en_ola = [p for p in indices_picos if inicio_idx <= p <␣
 ↪fin_idx]
        if pico_idx_en_ola:
            pico_idx = pico_idx_en_ola[0]
            olas_con_hitos.append({
                'nombre': nombres_olas_base[i], 'inicio_fecha': datos_df.
 ↪index[inicio_idx],
                'fin_fecha': datos_df.index[fin_idx], 'pico_fecha': datos_df.
 ↪index[pico_idx],
                'pico_valor': datos_df['casos_suavizados'].iloc[pico_idx],
                'valle_valor': datos_df['casos_suavizados'].iloc[inicio_idx],
                'color': PALETA[f'ola_{i+1}']
            })

# ------------------------------------------------------------------------
# 4. CREACIÓN DE LA GRÁFICA DE DOS PANELES HORIZONTALES
# ------------------------------------------------------------------------
print("Generando Figura 3.1 con ajustes finales de tamaño y anotaciones...")
# --- AJUSTE: Se aumenta el tamaño de la figura ---
fig_3_1, (ax_lin, ax_log) = plt.subplots(
    nrows=1, ncols=2, figsize=(24, 10), sharex=True
```

```
)
fig_3_1.suptitle(r'\textbf{Figura 3.1: Dinámica y Hitos de la Pandemia de␣
 ↪COVID-19 en México (5 Olas)}', y=1.0)

axes = [ax_lin, ax_log]
scales = ['linear', 'log']
titles = [r'\textbf{Panel A: Dinámica en Escala Lineal}', r'\textbf{Panel B:␣
 ↪Dinámica en Escala Logarítmica}']

for ax, scale, title in zip(axes, scales, titles):
    ax.set_title(title)
    ax.set_yscale(scale)
    ax.set_xlim(datos_df.index.min(), datos_df.index.max())

    for ola in olas_con_hitos:
        ax.axvspan(ola['inicio_fecha'], ola['fin_fecha'], color=ola['color'],␣
 ↪alpha=0.15, ec='none')
    ax.plot(datos_df.index, datos_df['casos_suavizados'],␣
 ↪color=PALETA['datos'], lw=2.5)

    for i, ola in enumerate(olas_con_hitos):
        ax.annotate(f"Pico {i+1}", xy=(ola['pico_fecha'], ola['pico_valor']),
                    xytext=(0, 15), textcoords='offset points', ha='center',␣
 ↪fontsize=10,
                    arrowprops=dict(arrowstyle="->", color=PALETA['pico']),␣
 ↪color=PALETA['pico'],
                    bbox=dict(boxstyle="round,pad=0.2", fc=PALETA['fondo'],␣
 ↪ec='none', alpha=0.7))
        ax.annotate(f"Inicio {i+1}", xy=(ola['inicio_fecha'],␣
 ↪ola['valle_valor']),
                    xytext=(-5, -25), textcoords='offset points', ha='right',␣
 ↪fontsize=10,
                    arrowprops=dict(arrowstyle="->", color=PALETA['valle']),␣
 ↪color=PALETA['valle'],
                    bbox=dict(boxstyle="round,pad=0.2", fc=PALETA['fondo'],␣
 ↪ec='none', alpha=0.7))

    # --- AJUSTE: Lógica de anotación ahora usa los niveles manuales ---
    for evento_dict in EVENTOS_CLAVE:
        fecha = pd.to_datetime(evento_dict['fecha'])
        nivel = evento_dict.get('nivel', 0) # Usa nivel 0 por defecto
        if datos_df.index.min() <= fecha <= datos_df.index.max():
            y_curve_pos = datos_df['casos_suavizados'].asof(fecha)
            if pd.isna(y_curve_pos): continue

            y_max = ax.get_ylim()[1]
```

```
            y_text_pos = y_max * (0.9 - (nivel * 0.12)) # Usamos el 'nivel' para la altura

            ax.annotate(evento_dict['texto'], xy=(fecha, y_curve_pos), xytext=(fecha, y_text_pos),
                        fontsize=10, ha='center', va='bottom', color=PALETA['texto_principal'],
                        bbox=dict(boxstyle="round,pad=0.3", fc=PALETA['fondo'], ec='none', alpha=0.8),
                        arrowprops=dict(arrowstyle="-", connectionstyle="angle3,angleA=0,angleB=90",
                                        color=PALETA['anotacion_politica'], lw=1.0, ls=':'))

ax_lin.set_ylabel('Número de Casos Diarios')
ax_lin.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, p: f'{int(x/1000):,}k'))
ax_lin.grid(True, which='major', axis='both', linestyle=':')

ax_log.set_ylabel('Número de Casos Diarios\n(Escala Logarítmica)')
ax_log.set_ylim(bottom=100)
ax_log.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, p: f'{int(x/1000):,}k' if x >= 1000 else str(int(x))))
ax_log.grid(True, which='major', axis='both', linestyle=':')

ax_log.set_xlabel('Fecha')
ax_log.xaxis.set_major_locator(mdates.MonthLocator(interval=4))
ax_log.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
plt.setp(ax_log.get_xticklabels(), rotation=30, ha="right")

ola_handles = [Patch(facecolor=ola['color'], alpha=0.3, label=ola['nombre']) for ola in olas_con_hitos]
fig_3_1.legend(handles=ola_handles, loc='lower center', bbox_to_anchor=(0.5, -0.1), ncol=5, title='Periodos de Olas Epidémicas')

fig_3_1.tight_layout(rect=[0, 0.05, 1, 0.95])

ruta_figura_pdf = 'Figuras Tesis/Figura_3_1_Panel_Horizontal_Final.pdf'
ruta_figura_png = 'Figuras Tesis/Figura_3_1_Panel_Horizontal_Final.png'
fig_3_1.savefig(ruta_figura_pdf)
fig_3_1.savefig(ruta_figura_png)

plt.show()
print(f" Figura 3.1 (versión final con ajustes de tamaño y leyendas) generada y guardada.")
```

[ ]:
```
# ==============================================================================
# PASO 2: CELDA DE PARÁMETROS MAESTROS (VERSIÓN DEFINITIVA Y CONGELADA)
# Única fuente de verdad para todas las simulaciones posteriores.
# Actualizado el 6 de Agosto, 2025, con los valores finales del Monte Carlo.
# ==============================================================================

# Parámetros Fijos (D y FD)
D_PRE_OMICRON = 5.1
D_OMICRON = 3.0
FD_OLA_1_2 = 0.032
FD_OLA_3 = 0.054
FD_OLA_4_5 = 0.072

PARAMETROS_MAESTROS = {
    # Ola: {'nombre': 'Nombre Ola', 'R0': R0, 'S0_N': S0_N, 'epsilon':  , 'D':␣
 ↪Periodo Infeccioso, 'FD': Fracción Detección}
    1: {'nombre': 'Ola 1 (Original)',     'R0': 2.11, 'S0_N': 1.000, 'epsilon':␣
 ↪0.00, 'D': D_PRE_OMICRON, 'FD': FD_OLA_1_2},
    2: {'nombre': 'Ola 2 (Alpha/Gamma)',  'R0': 1.50, 'S0_N': 0.821, 'epsilon':␣
 ↪0.00, 'D': D_PRE_OMICRON, 'FD': FD_OLA_1_2},
    3: {'nombre': 'Ola 3 (Delta)',        'R0': 3.37, 'S0_N': 0.412, 'epsilon':␣
 ↪0.00, 'D': D_PRE_OMICRON, 'FD': FD_OLA_3},
    4: {'nombre': 'Ola 4 (Omicron BA.1)', 'R0': 4.28, 'S0_N': 0.593, 'epsilon':␣
 ↪0.50, 'D': D_OMICRON,     'FD': FD_OLA_4_5},
    # VALOR ACTUALIZADO AQUÍ ABAJO
    5: {'nombre': 'Ola 5 (Omicron Subs.)','R0': 3.36, 'S0_N': 0.397, 'epsilon':␣
 ↪0.00, 'D': D_OMICRON,     'FD': FD_OLA_4_5},
}

# Convertir a DataFrame para visualizar y confirmar
import pandas as pd
parametros_df = pd.DataFrame.from_dict(PARAMETROS_MAESTROS, orient='index')
print("--- Parámetros Maestros Definitivos (VERSIÓN CONGELADA) ---")
print(parametros_df)
```

[10]:
```
# ==============================================================================
# CELDA FINAL MEJORADA: FIGURA 3.2 - AJUSTE POR SUBREGISTRO (DISEÑO MINIMALISTA)
# ==============================================================================
# Este bloque de código es autocontenido y genera la versión final de la Figura␣
 ↪3.2
# con un diseño profesional y minimalista de alto impacto.
# ==============================================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from matplotlib.patches import Patch
```

```
from matplotlib.lines import Line2D  # <-- CORRECCIÓN: Se añade esta línea
from scipy.signal import find_peaks
import os
import locale

# --- Configuración del locale a español para meses ---
try:
    locale.setlocale(locale.LC_TIME, 'es_ES.UTF-8')
except locale.Error:
    try:
        locale.setlocale(locale.LC_TIME, 'Spanish_Spain')
    except locale.Error:
        print("Locale en español no encontrado. Los meses pueden aparecer en␣
 ↪inglés.")

# ------------------------------------------------------------------------
# 1. CONFIGURACIÓN (PALETA, ESTILO Y DATOS CLAVE)
# ------------------------------------------------------------------------
PALETA = {
    'datos_reales': '#0072B2', 'datos_reportados': '#7f7f7f',
    'ola_1': '#2a9d8f', 'ola_2': '#e9c46a', 'ola_3': '#f4a261',
    'ola_4': '#e76f51', 'ola_5': '#d62828',
    'texto_principal': '#333333', 'rejilla': '#EAEAEA', 'fondo': '#FFFFFF',
    'ensanut': '#f0f0f0'
}
FRACCIONES_DETECCION = {
    'Ola 1 (Original)': 0.032, 'Ola 2 (Alpha/Gamma)': 0.032, 'Ola 3 (Delta)': 0.
 ↪054,
    'Ola 4 (Ómicron BA.1)': 0.072, 'Ola 5 (Ómicron Subs.)': 0.072
}
PERIODOS_ENSANUT = [
    {'nombre': 'ENSANUT 2020', 'inicio': '2020-08-17', 'fin': '2020-11-20'},
    {'nombre': 'ENSANUT 2021', 'inicio': '2021-08-16', 'fin': '2021-11-30'},
    {'nombre': 'ENSANUT 2022', 'inicio': '2022-08-02', 'fin': '2022-11-14'}
]
plt.rcParams.update({
    'font.family': 'serif', 'font.serif': ['Computer Modern Roman'], 'text.
 ↪usetex': True,
    'text.latex.preamble': r'\usepackage{amsmath}', 'figure.titlesize': 20,
    'axes.titlesize': 16, 'axes.labelsize': 14, 'xtick.labelsize': 12,
    'ytick.labelsize': 12, 'legend.fontsize': 12, 'axes.linewidth': 1.0,
    'lines.linewidth': 2.5, 'figure.facecolor': PALETA['fondo'],
    'axes.facecolor': PALETA['fondo'], 'axes.labelcolor':␣
 ↪PALETA['texto_principal'],
    'xtick.color': PALETA['texto_principal'], 'ytick.color':␣
 ↪PALETA['texto_principal'],
```

```
    'grid.color': PALETA['rejilla'], 'grid.linestyle': '-', 'axes.spines.top':␣
 ↪False,
    'axes.spines.right': False, 'figure.dpi': 300, 'savefig.dpi': 300, 'savefig.
 ↪bbox': 'tight'
})
if not os.path.exists('Figuras Tesis'):
    os.makedirs('Figuras Tesis')

# ------------------------------------------------------------------------------
# 2. CARGA Y PROCESAMIENTO DE DATOS (DEFINICIÓN DE 5 OLAS)
# ------------------------------------------------------------------------------
nombre_archivo = 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv'
print(f"Cargando datos desde '{nombre_archivo}'...")
df_raw = pd.read_csv(nombre_archivo)
df_nacional = df_raw[df_raw['nombre'] == 'Nacional'].copy()
casos_diarios_raw = df_nacional.iloc[:, 3:].astype(float).T
datos_df = pd.DataFrame(index=pd.to_datetime(casos_diarios_raw.index,␣
 ↪format='%d-%m-%Y'))
datos_df['casos_suavizados'] = datos_df.assign(casos_crudos=casos_diarios_raw.
 ↪values)['casos_crudos'].rolling(window=7, center=True, min_periods=1).mean()
print("Datos cargados y procesados.")

indices_valles, _ = find_peaks(-datos_df['casos_suavizados'], distance=100,␣
 ↪prominence=2500)
fronteras = np.unique(np.concatenate(([0], indices_valles, [len(datos_df)-1])))
nombres_olas_base = list(FRACCIONES_DETECCION.keys())
olas_periodos = []
for i in range(len(fronteras) - 1):
    if i < len(nombres_olas_base):
        olas_periodos.append({
            'nombre': nombres_olas_base[i], 'inicio': datos_df.
 ↪index[fronteras[i]],
            'fin': datos_df.index[fronteras[i+1]], 'color': PALETA[f'ola_{i+1}']
        })

# ------------------------------------------------------------------------------
# 3. CÁLCULO DE CASOS AJUSTADOS
# ------------------------------------------------------------------------------
datos_df['casos_ajustados'] = np.nan
for ola in olas_periodos:
    fraccion = FRACCIONES_DETECCION[ola['nombre']]
    mask = (datos_df.index >= ola['inicio']) & (datos_df.index < ola['fin'])
    datos_df.loc[mask, 'casos_ajustados'] = datos_df.loc[mask,␣
 ↪'casos_suavizados'] / fraccion
datos_df['casos_ajustados'] = datos_df['casos_ajustados'].ffill()
print("Cálculo de casos ajustados por subregistro completado.")
```

```
# ------------------------------------------------------------------------------
# 4. CREACIÓN DE LA GRÁFICA DE DOS PANELES HORIZONTALES
# ------------------------------------------------------------------------------
print("Generando Figura 3.2 con diseño minimalista...")
fig_3_2, (ax_lin, ax_log) = plt.subplots(
    nrows=1, ncols=2, figsize=(22, 9), sharex=True
)
fig_3_2.suptitle(r'\textbf{Figura 3.2: Comparativa de Casos Reportados vs.␣
 ↪Infecciones Estimadas por Subregistro}', y=1.0)

axes = [ax_lin, ax_log]
scales = ['linear', 'log']
titles = [r'\textbf{Panel A: Escala Lineal}', r'\textbf{Panel B: Escala␣
 ↪Logarítmica}']

for ax, scale, title in zip(axes, scales, titles):
    ax.set_title(title, pad=15)
    ax.set_yscale(scale)
    ax.set_xlim(datos_df.index.min() - pd.Timedelta(days=10), datos_df.index.
 ↪max() + pd.Timedelta(days=10))

    # Capa 1: Periodos ENSANUT (más sutil)
    for encuesta in PERIODOS_ENSANUT:
        ax.axvspan(pd.to_datetime(encuesta['inicio']), pd.
 ↪to_datetime(encuesta['fin']),
                   color=PALETA['ensanut'], alpha=0.6, zorder=0, ec='grey',␣
 ↪hatch='//')

    # Capa 2: Curvas de casos
    ax.plot(datos_df.index, datos_df['casos_suavizados'],␣
 ↪color=PALETA['datos_reportados'], lw=2, label='Casos Reportados␣
 ↪(suavizados)')
    ax.plot(datos_df.index, datos_df['casos_ajustados'],␣
 ↪color=PALETA['datos_reales'], lw=2.5, label='Infecciones Reales (estimadas)')

    # Capa 3: Anotaciones de Olas y Detección (nuevo sistema)
    y_max_ax = ax.get_ylim()[1]
    for i, ola in enumerate(olas_periodos):
        fecha_central = ola['inicio'] + (ola['fin'] - ola['inicio']) / 2
        fraccion = FRACCIONES_DETECCION[ola['nombre']]

        # Posición del texto de la anotación
        if scale == 'log':
            # Manejar posibles valores no positivos en el eje y antes de tomar␣
 ↪el logaritmo
```

```
            y_min, y_max = ax.get_ylim()
            if y_min <= 0: y_min = 1 # Establecer un mínimo seguro si es␣
 ↪necesario
            y_max_log = np.log10(y_max)
            y_min_log = np.log10(y_min)
            y_text_pos = 10**(y_min_log + (y_max_log - y_min_log) * (0.85 -␣
 ↪(i%2)*0.13))
        else:
            y_text_pos = ax.get_ylim()[1] * (0.85 - (i%2)*0.13)

        # Dibujar la anotación principal
        anotacion = ax.text(fecha_central, y_text_pos,␣
 ↪f"\\textbf{{{ola['nombre']}}}\nFracción de Detección: {fraccion*100:.1f}\\%",
                            ha='center', va='center', fontsize=10,
                            bbox=dict(boxstyle="round,pad=0.4",␣
 ↪fc=PALETA['fondo'], ec=ola['color'], alpha=0.9))

        # Dibujar líneas guía al inicio y fin de la ola
        ax.plot([ola['inicio'], fecha_central], [y_text_pos*0.9, y_text_pos],␣
 ↪color=ola['color'], lw=0.8, ls=':')
        ax.plot([ola['fin'], fecha_central], [y_text_pos*0.9, y_text_pos],␣
 ↪color=ola['color'], lw=0.8, ls=':')

# Formato específico del panel LINEAL (ax_lin)
ax_lin.set_ylabel('Número de Casos Diarios')
ax_lin.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, p: f'{int(x/
 ↪1000):,}k' if x > 0 else '0'))
ax_lin.grid(True, which='major', axis='both')

# Formato específico del panel LOGARÍTMICO (ax_log)
ax_log.set_ylabel('Número de Casos Diarios (Escala Logarítmica)')
ax_log.set_ylim(bottom=100)
ax_log.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, p: f'{int(x/
 ↪1000):,}k' if x >= 1000 else str(int(x))))
ax_log.grid(True, which='major', axis='both')

# Formato del eje X (compartido)
ax_lin.set_xlabel('Fecha')
ax_log.set_xlabel('Fecha')
ax_log.xaxis.set_major_locator(mdates.MonthLocator(interval=4))
ax_log.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
plt.setp(ax_log.get_xticklabels(), rotation=0, ha="center")

# --- Leyenda Simplificada ---
handles = [
```

```
    Line2D([0], [0], color=PALETA['datos_reportados'], lw=2.5, label='Casos␣
 ↪Reportados'),
    Line2D([0], [0], color=PALETA['datos_reales'], lw=2.5, label='Infecciones␣
 ↪Estimadas'),
    Patch(facecolor=PALETA['ensanut'], alpha=0.6, hatch='//', label='Periodo␣
 ↪Encuesta ENSANUT')
]
ax_lin.legend(handles=handles, loc='upper left')
ax_log.legend(handles=handles, loc='upper left')

# Ajuste final
fig_3_2.tight_layout(rect=[0, 0, 1, 0.95])

# --- Guardado ---
ruta_figura_pdf = 'Figuras Tesis/Figura_3_2_Subregistro_Minimalista.pdf'
ruta_figura_png = 'Figuras Tesis/Figura_3_2_Subregistro_Minimalista.png'
fig_3_2.savefig(ruta_figura_pdf)
fig_3_2.savefig(ruta_figura_png)

plt.show()
print(f" Figura 3.2 (versión minimalista y profesional) generada y guardada.")
```

```
Cargando datos desde 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv'…
Datos cargados y procesados.
Cálculo de casos ajustados por subregistro completado.
Generando Figura 3.2 con diseño minimalista…
```

Figura 3.2: Comparativa de Casos Reportados vs. Infecciones Estimadas por Subregistro

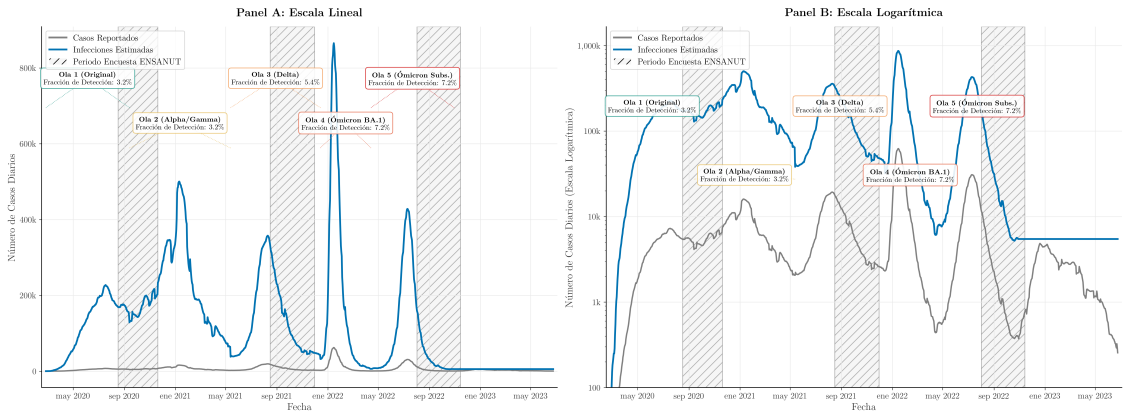


Figura 3.2 (versión minimalista y profesional) generada y guardada.

[11]:
```
# ================================================================================
# CELDA FINAL: CÁLCULO DE SO/N POR OPTIMIZACIÓN Y GENERACIÓN DE TABLA 3.1
# ================================================================================
```

```python
# Este bloque de código es autocontenido.
# ==============================================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from scipy.integrate import solve_ivp
from sklearn.metrics import mean_squared_error
import os

# ------------------------------------------------------------------------------
# 1. CONFIGURACIÓN Y DATOS DE ENTRADA COMPLETOS
# ------------------------------------------------------------------------------
PALETA = {
    'header': '#0d3b66', 'fondo': '#FFFFFF', 'texto_principal': '#333333',
    'ola_1': '#2a9d8f', 'ola_2': '#e9c46a', 'ola_3': '#f4a261',
    'ola_4': '#e76f51', 'ola_5': '#d62828'
}
plt.rcParams.update({
    'font.family': 'serif', 'font.serif': ['Computer Modern Roman'], 'text.
 ↪usetex': True,
    'text.latex.preamble': r'\usepackage{amsmath}\usepackage{amssymb}',
    'figure.dpi': 300, 'savefig.dpi': 300, 'savefig.bbox': 'tight'
})

# Parámetros base con R0 iniciales y D definitivos
PARAMETROS_BASE = [
    {'Ola': 'Ola 1 (Original)', 'Variante Dominante': 'Linajes Ancestrales',␣
 ↪'D': 10, 'D_fuente': '(Gries, et al., 2021)', 'Fraccion_Deteccion': 0.032,␣
 ↪'FD_fuente': '(ENSANUT 2020)', 'R0_inicial': 1.92},
    {'Ola': 'Ola 2 (Alpha/Gamma)', 'Variante Dominante': 'Alpha, Gamma', 'D':␣
 ↪11, 'D_fuente': '(Rodríguez-Morales, et al., 2021)', 'Fraccion_Deteccion': 0.
 ↪032, 'FD_fuente': '(Extrapolado)', 'R0_inicial': 1.65},
    {'Ola': 'Ola 3 (Delta)', 'Variante Dominante': 'Delta (B.1.617.2)', 'D': 9,␣
 ↪'D_fuente': '(Singh, et al., 2021)', 'Fraccion_Deteccion': 0.054,␣
 ↪'FD_fuente': '(ENSANUT 2021)', 'R0_inicial': 2.84},
    {'Ola': 'Ola 4 (Ómicron BA.1)', 'Variante Dominante': 'Ómicron (BA.1)', 'D':
 ↪ 7, 'D_fuente': '(Lee, et al., 2022)', 'Fraccion_Deteccion': 0.072,␣
 ↪'FD_fuente': '(ENSANUT 2022)', 'R0_inicial': 1.82},
    {'Ola': 'Ola 5 (Ómicron Subs.)', 'Variante Dominante': 'Ómicron (BA.4/5, BQ.
 ↪1)', 'D': 6, 'D_fuente': '(Iwata, et al., 2023)', 'Fraccion_Deteccion': 0.
 ↪072, 'FD_fuente': '(ENSANUT 2022)', 'R0_inicial': 5.41} # Usamos el R0 de la␣
 ↪Ola 5 del análisis de 6 olas como el más representativo
]
SON_FIJOS = [1.0, 0.821, 0.412] # Valores fijos para las primeras 3 olas
```

```
if not os.path.exists('Figuras Tesis'):
    os.makedirs('Figuras Tesis')

# ------------------------------------------------------------------------------
# 2. FUNCIONES PARA SIMULACIÓN Y OPTIMIZACIÓN
# ------------------------------------------------------------------------------
def sir_model(t, y, R0, gamma, N):
    S, I = y[0], y[1]
    beta = R0 * gamma
    dSdt = -beta * S * I / N
    dIdt = beta * S * I / N - gamma * I
    return [dSdt, dIdt]

# ------------------------------------------------------------------------------
# 3. CÁLCULO DE S0/N CON LÓGICA HÍBRIDA (FIJA + OPTIMIZACIÓN DE EPSILON)
# ------------------------------------------------------------------------------
print("Calculando S0/N para cada ola (método híbrido)...")
nombre_archivo = 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv'
df_raw = pd.read_csv(nombre_archivo)
df_nacional = df_raw[df_raw['nombre'] == 'Nacional'].copy()
poblacion_mexico = df_nacional['poblacion'].iloc[0]
casos_diarios_raw = df_nacional.iloc[:, 3:].astype(float).T
datos_df = pd.DataFrame(index=pd.to_datetime(casos_diarios_raw.index,␣
 ↪format='%d-%m-%Y'))
datos_df['casos_suavizados'] = datos_df.assign(casos_crudos=casos_diarios_raw.
 ↪values)['casos_crudos'].rolling(window=7, center=True, min_periods=1).mean()

indices_valles, _ = find_peaks(-datos_df['casos_suavizados'], distance=100,␣
 ↪prominence=2500)
fronteras = np.unique(np.concatenate(([0], indices_valles, [len(datos_df)-1])))

s_frac_teorico = 1.0 # Fracción susceptible si no hubiera evasión inmune

for i in range(len(PARAMETROS_BASE)):
    ola_params = PARAMETROS_BASE[i]
    inicio_ola = datos_df.index[fronteras[i]]
    fin_ola = datos_df.index[fronteras[i+1]] if i < len(fronteras) - 1 else␣
 ↪datos_df.index[-1]
    mask_ola = (datos_df.index >= inicio_ola) & (datos_df.index < fin_ola)

    # --- Asignación de S0/N ---
    if i < 3: # Olas 1, 2, 3 usan valores fijos
        s0_actual = S0N_FIJOS[i]
        ola_params['S0_N'] = s0_actual
        ola_params['epsilon'] = 0
        ola_params['S0_N_justificacion'] = f'(Valor fijo para␣
 ↪{ola_params["Variante Dominante"]})'
```

```
        if i == 0: ola_params['S0_N_justificacion'] = '(Población sin␣
↪inmunidad)'
        print(f"{ola_params['Ola']}: S0/N fijado en {s0_actual:.3f}")

    else: # Olas 4 y 5 (Ómicron) requieren optimización de epsilon
        print(f"--- Optimizando epsilon para {ola_params['Ola']} ---")

        # Preparar datos reales para comparar
        casos_reales_ola = (datos_df.loc[mask_ola, 'casos_suavizados'] /␣
↪ola_params['Fraccion_Deteccion']).values
        infectados_reales_frac = casos_reales_ola / poblacion_mexico

        r_frac_teorico = 1.0 - s_frac_teorico
        gamma = 1.0 / ola_params['D']
        R0 = ola_params['R0_inicial']

        best_epsilon, min_rmse = -1, np.inf

        # Barrido de parámetros para epsilon
        for epsilon_candidate in np.linspace(0, 1, 101):
            s0_candidate = s_frac_teorico + r_frac_teorico * epsilon_candidate
            if s0_candidate > 1: s0_candidate = 1.0

            # Simulación SIR
            sol = solve_ivp(sir_model, (0, len(casos_reales_ola)),␣
↪[s0_candidate * poblacion_mexico, casos_reales_ola[0]],
                            t_eval=np.arange(len(casos_reales_ola)),
                            args=(R0, gamma, poblacion_mexico))

            infectados_simulados_frac = sol.y[1] / poblacion_mexico

            # Calcular error
            rmse = np.sqrt(mean_squared_error(infectados_reales_frac,␣
↪infectados_simulados_frac))
            if rmse < min_rmse:
                min_rmse = rmse
                best_epsilon = epsilon_candidate

        s0_actual = s_frac_teorico + r_frac_teorico * best_epsilon
        ola_params['S0_N'] = s0_actual
        ola_params['epsilon'] = best_epsilon
        ola_params['S0_N_justificacion'] = f'($\\epsilon={best_epsilon:.2f}$␣
↪por ajuste a datos)'
        print(f"-> Epsilon óptimo encontrado: {best_epsilon:.2f} => S0/N =␣
↪{s0_actual:.3f}")
```

```
    # Actualizar s_frac_teorico para la siguiente ola
    casos_reportados_ola = datos_df.loc[mask_ola, 'casos_suavizados'].sum()
    infecciones_reales_ola = casos_reportados_ola /␣
 ↪ola_params['Fraccion_Deteccion']
    s_frac_teorico = s0_actual - (infecciones_reales_ola / poblacion_mexico)
    if s_frac_teorico < 0: s_frac_teorico = 0

# ------------------------------------------------------------------------------
# 5. PREPARACIÓN DEL DATAFRAME FINAL Y GENERACIÓN DE TABLA
# ------------------------------------------------------------------------------
tabla_data = []
for p in PARAMETROS_BASE:
    tabla_data.append({
        r'\textbf{Ola Epidémica}': f"\\textbf{{{p['Ola']}}}",
        r'\textbf{Variante Dominante}': p['Variante Dominante'],
        r'\textbf{Periodo Infeccioso (D)}': f"{p['D']}␣
 ↪días\n\\textit{{\\scriptsize {p['D_fuente']}}}",
        r'\textbf{Fracción Susceptible ($S_0/N$)}': f"{p['S0_N']:.
 ↪3f}\n\\textit{{\\scriptsize {p.get('S0_N_justificacion', '')}}}",
        r'\textbf{Detección (Fracción / Factor)}':␣
 ↪f"{p['Fraccion_Deteccion']*100:.1f}%% / {1/p['Fraccion_Deteccion']:.
 ↪1f}\n\\textit{{\\scriptsize {p['FD_fuente']}}}"
    })
df_tabla = pd.DataFrame(tabla_data)

print("\n--- Generando Tabla 3.1 como imagen ---")
fig_tabla, ax = plt.subplots(figsize=(14, 4))
ax.axis('off')
tabla = ax.table(cellText=df_tabla.values, colLabels=df_tabla.columns,
                 cellLoc='center', loc='center', colWidths=[0.18, 0.22, 0.15, 0.
 ↪22, 0.23])
tabla.auto_set_font_size(False); tabla.set_fontsize(9); tabla.scale(1, 2.5)
for (i, j), cell in tabla.get_celld().items():
    if i == 0:
        cell.set_text_props(weight='bold', color='white'); cell.
 ↪set_facecolor(PALETA['header'])
    else:
        cell.set_facecolor(PALETA['fondo']);
        if j == 0: cell.get_text().set_color(PALETA[f'ola_{i}'])
    cell.set_edgecolor('lightgrey')
fig_tabla.suptitle(r'\textbf{Tabla 3.1: Parámetros Epidemiológicos por Ola (con␣
 ↪Evasión Inmune)}', y=1.05)
plt.tight_layout()

ruta_tabla_pdf = 'Figuras Tesis/Tabla_3_1_Parametros_Final.pdf'
ruta_tabla_png = 'Figuras Tesis/Tabla_3_1_Parametros_Final.png'
```

```
fig_tabla.savefig(ruta_tabla_pdf); fig_tabla.savefig(ruta_tabla_png)
plt.show()

print(f"  Tabla 3.1 (versión final y robusta) generada y guardada.")
```

```
Calculando S0/N para cada ola (método híbrido)…
Ola 1 (Original): S0/N fijado en 1.000
Ola 2 (Alpha/Gamma): S0/N fijado en 0.821
Ola 3 (Delta): S0/N fijado en 0.412
--- Optimizando epsilon para Ola 4 (Ómicron BA.1) ---
-> Epsilon óptimo encontrado: 0.50 => S0/N = 0.593
--- Optimizando epsilon para Ola 5 (Ómicron Subs.) ---
-> Epsilon óptimo encontrado: 0.00 => S0/N = 0.397

--- Generando Tabla 3.1 como imagen ---
```

Tabla 3.1: Parámetros Epidemiológicos por Ola (con Evasión Inmune)

Ola Epidémica	Variante Dominante	Periodo Infeccioso (D)	Fracción Susceptible (S_0/N)	Detección (Fracción / Factor)
Ola 1 (Original)	Linajes Ancestrales	10 días *(Gries, et al., 2021)*	1.000 *(Población sin inmunidad)*	3.2 *(ENSANUT 2020)*
Ola 2 (Alpha/Gamma)	Alpha, Gamma	11 días *(Rodríguez-Morales, et al., 2021)*	0.821 *(Valor fijo para Alpha, Gamma)*	3.2 *(Extrapolado)*
Ola 3 (Delta)	Delta (B.1.617.2)	9 días *(Singh, et al., 2021)*	0.412 *(Valor fijo para Delta (B.1.617.2))*	5.4 *(ENSANUT 2021)*
Ola 4 (Ómicron BA.1)	Ómicron (BA.1)	7 días *(Lee, et al., 2022)*	0.593 *($\epsilon = 0.50$ por ajuste a datos)*	7.2 *(ENSANUT 2022)*
Ola 5 (Ómicron Subs.)	Ómicron (BA.4/5, BQ.1)	6 días *(Iwata, et al., 2023)*	0.397 *($\epsilon = 0.00$ por ajuste a datos)*	7.2 *(ENSANUT 2022)*

```
Tabla 3.1 (versión final y robusta) generada y guardada.
```

[12]:
```
# ============================================================================
# CELDA CORREGIDA: GENERA TABLA 4.1 (IMAGEN) Y FIGURA 4.1
# ============================================================================
# Este bloque de código es autocontenido y genera tanto la tabla de resultados␣
 ↪como
# la figura de barras en formato de imagen de alta calidad.
# ============================================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from scipy import stats, optimize
from tqdm import tqdm
import os
import locale

# --- Configuración del locale a español para meses ---
```

```
try:
    locale.setlocale(locale.LC_TIME, 'es_ES.UTF-8')
except locale.Error:
    try:
        locale.setlocale(locale.LC_TIME, 'Spanish_Spain')
    except locale.Error:
        print("Locale en español no encontrado. Los meses pueden aparecer en␣
 ↪inglés.")

# ------------------------------------------------------------------------
# 1. CONFIGURACIÓN (PALETA, ESTILO Y PARÁMETROS FINALES)
# ------------------------------------------------------------------------
PALETA = {
    'header': '#0d3b66', 'ola_1': '#2a9d8f', 'ola_2': '#e9c46a',
    'ola_3': '#f4a261', 'ola_4': '#e76f51', 'ola_5': '#d62828',
    'texto_principal': '#333333', 'rejilla': '#EAEAEA', 'fondo': '#FFFFFF'
}
plt.rcParams.update({
    'font.family': 'serif', 'font.serif': ['Computer Modern Roman'], 'text.
 ↪usetex': True,
    'text.latex.preamble': r'\usepackage{amsmath}\usepackage{amssymb}',
    'figure.titlesize': 18, 'axes.titlesize': 14, 'axes.labelsize': 12,
    'xtick.labelsize': 11, 'ytick.labelsize': 11, 'legend.fontsize': 11,
    'axes.linewidth': 1.0, 'lines.linewidth': 2.5, 'figure.facecolor':␣
 ↪PALETA['fondo'],
    'axes.facecolor': PALETA['fondo'], 'axes.labelcolor':␣
 ↪PALETA['texto_principal'],
    'xtick.color': PALETA['texto_principal'], 'ytick.color':␣
 ↪PALETA['texto_principal'],
    'grid.color': PALETA['rejilla'], 'grid.linestyle': '--', 'axes.spines.top':␣
 ↪False,
    'axes.spines.right': False, 'figure.dpi': 300, 'savefig.dpi': 300, 'savefig.
 ↪bbox': 'tight'
})

PARAMETROS_FINALES = [
    {'Ola': 'Ola 1 (Original)',       'D': 10, 'Fraccion_Deteccion': 0.032,␣
 ↪'S0_N': 1.000},
    {'Ola': 'Ola 2 (Alpha/Gamma)',    'D': 11, 'Fraccion_Deteccion': 0.032,␣
 ↪'S0_N': 0.821},
    {'Ola': 'Ola 3 (Delta)',          'D': 9,  'Fraccion_Deteccion': 0.054,␣
 ↪'S0_N': 0.412},
    {'Ola': 'Ola 4 (Ómicron BA.1)',   'D': 7,  'Fraccion_Deteccion': 0.072,␣
 ↪'S0_N': 0.718},
    {'Ola': 'Ola 5 (Ómicron Subs.)', 'D': 6,  'Fraccion_Deteccion': 0.072,␣
 ↪'S0_N': 0.470}
```

```
]
CONFIG = {'N_SIM': 100000, 'INCERTIDUMBRE': 0.2}

if not os.path.exists('Figuras Tesis'):
    os.makedirs('Figuras Tesis')

# ------------------------------------------------------------------------------
# 2. CARGA DE DATOS Y DEFINICIÓN DE OLAS (5 OLAS)
# ------------------------------------------------------------------------------
nombre_archivo = 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv'
datos_df = pd.read_csv(
    nombre_archivo,
    index_col='nombre'
).loc['Nacional', 'poblacion':].iloc[1:].to_frame(name='casos_crudos')
datos_df.index = pd.to_datetime(datos_df.index, format='%d-%m-%Y')
datos_df['casos_suavizados'] = datos_df['casos_crudos'].rolling(7, center=True,␣
 ↪min_periods=1).mean()

indices_valles, _ = find_peaks(-datos_df['casos_suavizados'], distance=100,␣
 ↪prominence=2500)
fronteras = np.unique(np.concatenate(([0], indices_valles, [len(datos_df)-1])))
olas_definidas = {}
for i in range(len(fronteras) - 1):
    if i < len(PARAMETROS_FINALES):
        nombre_ola = PARAMETROS_FINALES[i]['Ola']
        olas_definidas[nombre_ola] = {
            'inicio': datos_df.index[fronteras[i]],
            'fin': datos_df.index[fronteras[i+1]]
        }

# ------------------------------------------------------------------------------
# 3. FUNCIONES DE AJUSTE
# ------------------------------------------------------------------------------
def encontrar_fase_exponencial(casos, window_size=21):
    if len(casos) < window_size: return 0, len(casos)
    log_casos = np.log(np.maximum(1, casos)); max_r2 = -1; best_start = 0
    for start in range(len(casos) - window_size):
        end = start + window_size; t = np.arange(window_size)
        slope, _, r_value, _, _ = stats.linregress(t, log_casos[start:end])
        if r_value**2 > max_r2 and slope > 0:
            max_r2 = r_value**2; best_start = start
    return best_start, best_start + window_size

def modelo_exponencial(t, casos, t_inf):
    if len(casos) < 2: return np.nan
    with np.errstate(divide='ignore'):
        log_casos = np.log(casos)
```

```
        log_casos[np.isneginf(log_casos)] = 0
    slope, _, r_value, _, _ = stats.linregress(t, log_casos)
    if np.isnan(slope): return np.nan
    r_eff = np.exp(slope * t_inf)
    return r_eff

# ------------------------------------------------------------------------------
# 4. CÁLCULO PRINCIPAL DE R0 (MONTE CARLO)
# ------------------------------------------------------------------------------
print(f"\nIniciando Simulación de Monte Carlo con {CONFIG['N_SIM']:,}␣
 ↪iteraciones...")
resultados_distribuciones = {}

def create_dist(mode, uncertainty):
    return {'min': mode * (1 - uncertainty), 'mode': mode, 'max': mode * (1 +␣
 ↪uncertainty)}

for i, params_ola in enumerate(PARAMETROS_FINALES):
    nombre_ola = params_ola['Ola']
    info = olas_definidas[nombre_ola]

    casos_ola = datos_df.loc[info['inicio']:info['fin'], 'casos_suavizados']
    start_idx, end_idx = encontrar_fase_exponencial(casos_ola.values)
    casos_exp = casos_ola.iloc[start_idx:end_idx]
    t_exp = np.arange(len(casos_exp))

    dist_D = create_dist(params_ola['D'], CONFIG['INCERTIDUMBRE'])
    dist_FD = create_dist(params_ola['Fraccion_Deteccion'],␣
 ↪CONFIG['INCERTIDUMBRE'])
    dist_S0 = create_dist(params_ola['S0_N'], CONFIG['INCERTIDUMBRE'])

    d_samples = np.random.triangular(dist_D['min'], dist_D['mode'],␣
 ↪dist_D['max'], CONFIG['N_SIM'])
    fd_samples = np.random.triangular(dist_FD['min'], dist_FD['mode'],␣
 ↪dist_FD['max'], CONFIG['N_SIM'])
    s0_n_samples = np.random.triangular(dist_S0['min'], dist_S0['mode'],␣
 ↪dist_S0['max'], CONFIG['N_SIM'])

    r0_results = []
    for j in tqdm(range(CONFIG['N_SIM']), desc=f"Procesando {nombre_ola}",␣
 ↪ncols=100):
        casos_corr = casos_exp.values / fd_samples[j]
        casos_corr[casos_corr == 0] = 1e-9
        r_eff = modelo_exponencial(t_exp, casos_corr, d_samples[j])
        if not np.isnan(r_eff):
            R0 = r_eff / s0_n_samples[j]
```

```
            r0_results.append(R0)

    resultados_distribuciones[nombre_ola] = np.array(r0_results)

# ------------------------------------------------------------------------------
# 5. GENERACIÓN DE TABLA 4.1 Y FIGURA 4.1
# ------------------------------------------------------------------------------
stats_data = []
for nombre, dist in resultados_distribuciones.items():
    stats_data.append({
        'Ola': nombre, 'R0 Medio': np.mean(dist),
        'IC_inf': np.percentile(dist, 2.5), 'IC_sup': np.percentile(dist, 97.5),
        'Desv. Est.': np.std(dist)
    })
df_stats = pd.DataFrame(stats_data)

# --- TABLA 4.1 (VERSIÓN TEXTO Y DATAFRAME PARA IMAGEN) ---
df_print = df_stats.copy()
df_print['Intervalo de Confianza 95%'] = df_print.apply(lambda row:␣
 ↪f'[{row["IC_inf"]:.2f} - {row["IC_sup"]:.2f}]', axis=1)
df_print_final = df_print[['Ola', 'R0 Medio', 'Desv. Est.', 'Intervalo de␣
 ↪Confianza 95%']].round(2)

print("\n" + "="*80)
print(r" Tabla 4.1: Estimaciones de R0 por Ola (Método de Monte Carlo)")
print("="*80)
print(df_print_final.to_string(index=False))
print("="*80)

# --- NUEVO: GENERACIÓN DE TABLA 4.1 COMO IMAGEN ---
print("\nGenerando Tabla 4.1 como imagen...")
fig_tabla, ax_tabla = plt.subplots(figsize=(10, 3))
ax_tabla.axis('off')

# Renombrar columnas para la tabla visual
df_tabla_visual = df_print_final.rename(columns={
    'Ola': r'\textbf{Ola Epidémica}',
    'R0 Medio': r'\textbf{$R_0$ (Media)}',
    'Desv. Est.': r'\textbf{$\sigma$}',
    'Intervalo de Confianza 95%': r'\textbf{IC 95\%}'
})

tabla = ax_tabla.table(cellText=df_tabla_visual.values,␣
 ↪colLabels=df_tabla_visual.columns,
                       cellLoc='center', loc='center', colWidths=[0.3, 0.15, 0.
 ↪15, 0.4])
tabla.auto_set_font_size(False); tabla.set_fontsize(10); tabla.scale(1, 2.5)
```

```
for (i, j), cell in tabla.get_celld().items():
    if i == 0:
        cell.set_text_props(weight='bold', color='white'); cell.
 ↪set_facecolor(PALETA['header'])
    else:
        cell.set_facecolor(PALETA['fondo']);
        if j == 0: cell.get_text().set_color(PALETA[f'ola_{i}'])
    cell.set_edgecolor('lightgrey')

fig_tabla.suptitle(r'\textbf{Tabla 4.1: Estimaciones de $R_0$ por Ola (Monte␣
 ↪Carlo)}', y=1.1)
plt.tight_layout()
ruta_tabla_pdf = 'Figuras Tesis/Tabla_4_1_Estimacion_R0.pdf'
ruta_tabla_png = 'Figuras Tesis/Tabla_4_1_Estimacion_R0.png'
fig_tabla.savefig(ruta_tabla_pdf); fig_tabla.savefig(ruta_tabla_png)
plt.show()
print(f"  Tabla 4.1 (versión imagen) generada y guardada.")


# --- FIGURA 4.1 ---
print("\nGenerando Figura 4.1...")
fig_4_1, ax = plt.subplots(figsize=(10, 6))
means = df_stats['R0 Medio']
errors = [means - df_stats['IC_inf'], df_stats['IC_sup'] - means]
colores = [PALETA[f'ola_{i+1}'] for i in range(len(df_stats))]
nombres_olas_grafico = [nombre.replace(' (', '\n(') for nombre in␣
 ↪df_stats['Ola']]

bars = ax.bar(nombres_olas_grafico, means, yerr=errors, color=colores,
              edgecolor='black', lw=0.8, capsize=5, zorder=10)
ax.axhline(1.0, color=PALETA['texto_principal'], linestyle='--', lw=1.5,␣
 ↪label=r'Umbral Epidémico ($R_0=1$)', zorder=5)

for i, bar in enumerate(bars):
    mean_val = df_stats['R0 Medio'].iloc[i]
    error_top_pos = df_stats['IC_sup'].iloc[i]
    ax.text(bar.get_x() + bar.get_width() / 2.0, error_top_pos + 0.05,
            f'{mean_val:.2f}', ha='center', va='bottom', weight='bold',␣
 ↪fontsize=11)

ax.set_title(r'\textbf{Figura 4.1: Estimación de $R_0$ con Intervalos de␣
 ↪Confianza del 95\%}')
ax.set_ylabel(r'Número Reproductivo Básico ($R_0$)')
ax.set_xlabel('Ola Epidémica')
ax.legend()
ax.grid(True, axis='y', zorder=0)
```

```
ax.set_ylim(bottom=0)
plt.tight_layout()

ruta_figura_pdf = 'Figuras Tesis/Figura_4_1_Estimacion_R0_MC.pdf'
ruta_figura_png = 'Figuras Tesis/Figura_4_1_Estimacion_R0_MC.png'
fig_4_1.savefig(ruta_figura_pdf)
fig_4_1.savefig(ruta_figura_png)

plt.show()
print(f"  Figura 4.1 generada y guardada.")
```

```
Iniciando Simulación de Monte Carlo con 100,000 iteraciones…
Procesando Ola 1 (Original): 100%|            | 100000/100000
[00:02<00:00, 33676.74it/s]
Procesando Ola 2 (Alpha/Gamma): 100%|          | 100000/100000
[00:03<00:00, 31405.40it/s]
Procesando Ola 3 (Delta): 100%|              | 100000/100000
[00:03<00:00, 30788.61it/s]
Procesando Ola 4 (Ómicron BA.1): 100%|          | 100000/100000
[00:03<00:00, 31500.96it/s]
Procesando Ola 5 (Ómicron Subs.): 100%|          | 100000/100000
[00:03<00:00, 30789.51it/s]


================================================================================
 Tabla 4.1: Estimaciones de R0 por Ola (Método de Monte Carlo)
================================================================================
                  Ola  R0 Medio  Desv. Est. Intervalo de Confianza 95%
     Ola 1 (Original)       2.12        0.22              [1.74 - 2.58]
  Ola 2 (Alpha/Gamma)       1.50        0.13              [1.29 - 1.77]
        Ola 3 (Delta)       3.37        0.29              [2.87 - 3.99]
 Ola 4 (Ómicron BA.1)       4.28        0.53              [3.36 - 5.41]
Ola 5 (Ómicron Subs.)       3.37        0.30              [2.84 - 4.01]
================================================================================

Generando Tabla 4.1 como imagen…
```

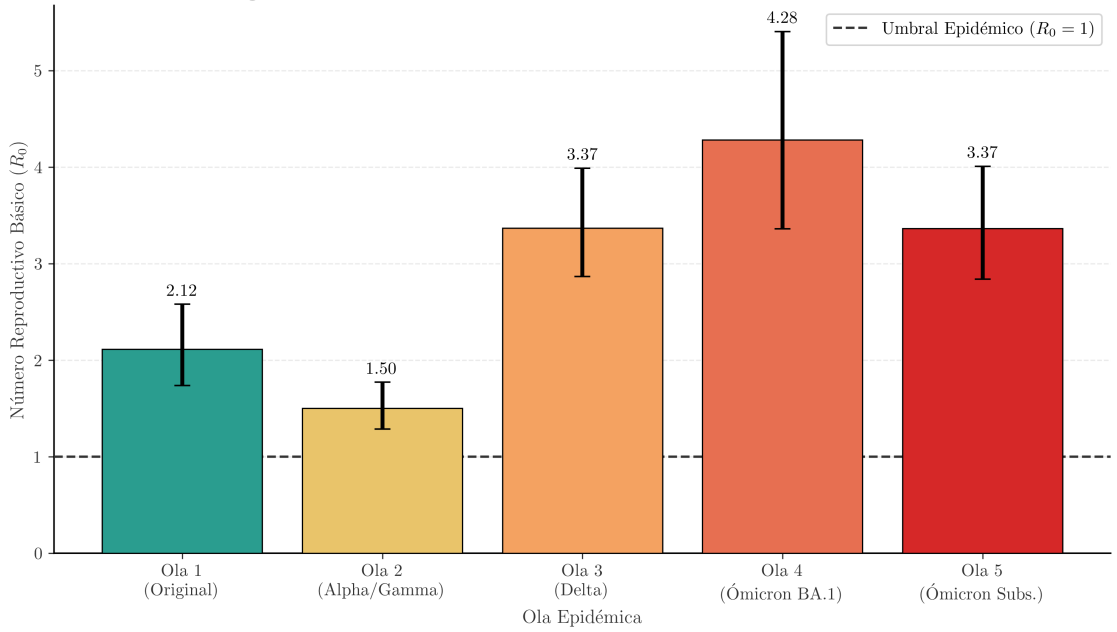
Tabla 4.1: Estimaciones de R_0 por Ola (Monte Carlo)

Ola Epidémica	R_0 (Media)	σ	IC 95%
Ola 1 (Original)	2.12	0.22	[1.74 - 2.58]
Ola 2 (Alpha/Gamma)	1.5	0.13	[1.29 - 1.77]
Ola 3 (Delta)	3.37	0.29	[2.87 - 3.99]
Ola 4 (Ómicron BA.1)	4.28	0.53	[3.36 - 5.41]
Ola 5 (Ómicron Subs.)	3.37	0.3	[2.84 - 4.01]

```
Tabla 4.1 (versión imagen) generada y guardada.
```

```
Generando Figura 4.1…
```

Figura 4.1: Estimación de R_0 con Intervalos de Confianza del 95%




```
Figura 4.1 generada y guardada.
```

[15]:
```
# ================================================================================
# CELDA PARA FIGURA 4.2: DISTRIBUCIONES DE PROBABILIDAD DE RO
# ================================================================================
# Este bloque de código es autocontenido. Ejecuta la simulación de Monte Carlo
# y genera la visualización de las distribuciones de probabilidad de RO.
# ================================================================================
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.signal import find_peaks
from scipy import stats
from tqdm import tqdm
import os
import locale

# --- Configuración del locale a español para meses ---
try:
    locale.setlocale(locale.LC_TIME, 'es_ES.UTF-8')
except locale.Error:
    try:
        locale.setlocale(locale.LC_TIME, 'Spanish_Spain')
    except locale.Error:
        print("Locale en español no encontrado. Los meses pueden aparecer en␣
 ↪inglés.")

# ------------------------------------------------------------------------
# 1. CONFIGURACIÓN (PALETA, ESTILO Y PARÁMETROS FINALES)
# ------------------------------------------------------------------------
PALETA = {
    'ola_1': '#2a9d8f', 'ola_2': '#e9c46a', 'ola_3': '#f4a261',
    'ola_4': '#e76f51', 'ola_5': '#d62828',
    'texto_principal': '#333333', 'rejilla': '#EAEAEA', 'fondo': '#FFFFFF'
}
plt.rcParams.update({
    'font.family': 'serif', 'font.serif': ['Computer Modern Roman'], 'text.
 ↪usetex': True,
    'text.latex.preamble': r'\usepackage{amsmath}', 'figure.titlesize': 18,
    'axes.titlesize': 14, 'axes.labelsize': 12, 'xtick.labelsize': 11,
    'ytick.labelsize': 11, 'legend.fontsize': 11, 'axes.linewidth': 1.0,
    'lines.linewidth': 2.0, 'figure.facecolor': PALETA['fondo'],
    'axes.facecolor': PALETA['fondo'], 'axes.labelcolor':␣
 ↪PALETA['texto_principal'],
    'xtick.color': PALETA['texto_principal'], 'ytick.color':␣
 ↪PALETA['texto_principal'],
    'grid.color': PALETA['rejilla'], 'grid.linestyle': '--', 'axes.spines.top':␣
 ↪False,
    'axes.spines.right': False, 'figure.dpi': 300, 'savefig.dpi': 300, 'savefig.
 ↪bbox': 'tight'
})

PARAMETROS_FINALES = [
```

```
    {'Ola': 'Ola 1 (Original)',       'D': 10, 'Fraccion_Deteccion': 0.032,␣
 ↪'S0_N': 1.000},
    {'Ola': 'Ola 2 (Alpha/Gamma)',    'D': 11, 'Fraccion_Deteccion': 0.032,␣
 ↪'S0_N': 0.821},
    {'Ola': 'Ola 3 (Delta)',          'D': 9,  'Fraccion_Deteccion': 0.054,␣
 ↪'S0_N': 0.412},
    {'Ola': 'Ola 4 (Ómicron BA.1)',   'D': 7,  'Fraccion_Deteccion': 0.072,␣
 ↪'S0_N': 0.718},
    {'Ola': 'Ola 5 (Ómicron Subs.)', 'D': 6,  'Fraccion_Deteccion': 0.072,␣
 ↪'S0_N': 0.470}
]
CONFIG = {'N_SIM': 100000, 'INCERTIDUMBRE': 0.2}

if not os.path.exists('Figuras Tesis'):
    os.makedirs('Figuras Tesis')

# ------------------------------------------------------------------------------
# 2. CARGA DE DATOS Y DEFINICIÓN DE OLAS (5 OLAS)
# ------------------------------------------------------------------------------
nombre_archivo = 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv'
datos_df = pd.read_csv(
    nombre_archivo,
    index_col='nombre'
).loc['Nacional', 'poblacion':].iloc[1:].to_frame(name='casos_crudos')
datos_df.index = pd.to_datetime(datos_df.index, format='%d-%m-%Y')
datos_df['casos_suavizados'] = datos_df['casos_crudos'].rolling(7, center=True,␣
 ↪min_periods=1).mean()

indices_valles, _ = find_peaks(-datos_df['casos_suavizados'], distance=100,␣
 ↪prominence=2500)
fronteras = np.unique(np.concatenate(([0], indices_valles, [len(datos_df)-1])))
olas_definidas = {}
for i in range(len(fronteras) - 1):
    if i < len(PARAMETROS_FINALES):
        nombre_ola = PARAMETROS_FINALES[i]['Ola']
        olas_definidas[nombre_ola] = {
            'inicio': datos_df.index[fronteras[i]],
            'fin': datos_df.index[fronteras[i+1]]
        }

# ------------------------------------------------------------------------------
# 3. FUNCIONES DE AJUSTE
# ------------------------------------------------------------------------------
def encontrar_fase_exponencial(casos, window_size=21):
    if len(casos) < window_size: return 0, len(casos)
    log_casos = np.log(np.maximum(1, casos)); max_r2 = -1; best_start = 0
```

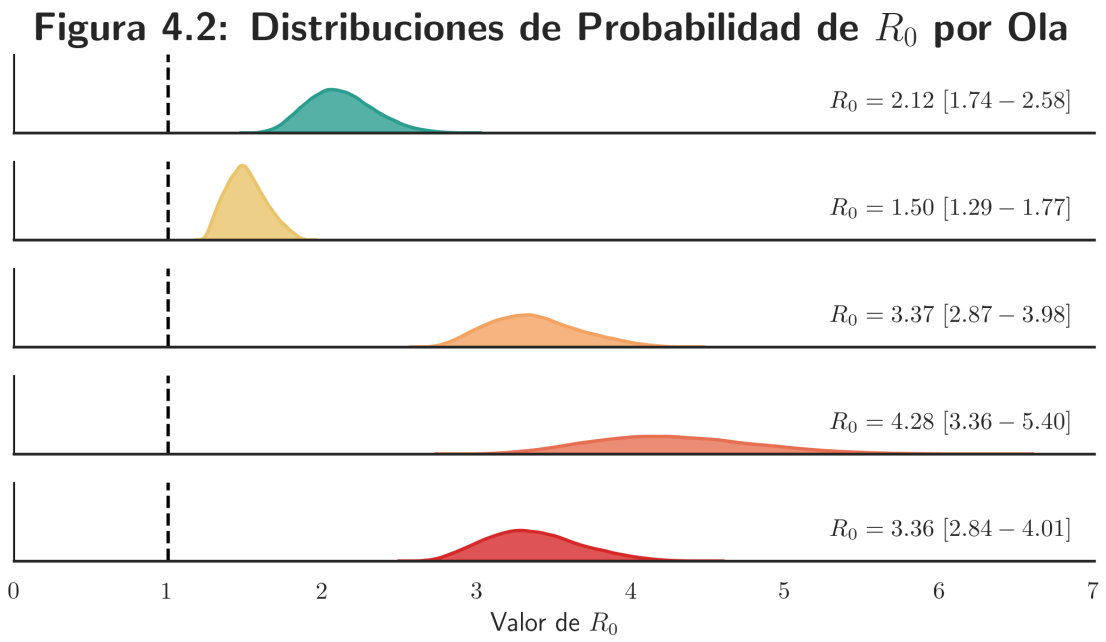
```
    for start in range(len(casos) - window_size):
        end = start + window_size; t = np.arange(window_size)
        slope, _, r_value, _, _ = stats.linregress(t, log_casos[start:end])
        if r_value**2 > max_r2 and slope > 0:
            max_r2 = r_value**2; best_start = start
    return best_start, best_start + window_size

def modelo_exponencial(t, casos, t_inf):
    if len(casos) < 2: return np.nan
    with np.errstate(divide='ignore'):
        log_casos = np.log(casos)
        log_casos[np.isneginf(log_casos)] = 0
    slope, _, r_value, _, _ = stats.linregress(t, log_casos)
    if np.isnan(slope): return np.nan
    r_eff = np.exp(slope * t_inf)
    return r_eff

# ------------------------------------------------------------------------------
# 4. CÁLCULO PRINCIPAL DE R0 (MONTE CARLO)
# ------------------------------------------------------------------------------
print(f"\nIniciando Simulación de Monte Carlo con {CONFIG['N_SIM']:,}␣
 ↪iteraciones...")
resultados_distribuciones = {}

def create_dist(mode, uncertainty):
    return {'min': mode * (1 - uncertainty), 'mode': mode, 'max': mode * (1 +␣
 ↪uncertainty)}

for i, params_ola in enumerate(PARAMETROS_FINALES):
    nombre_ola = params_ola['Ola']
    info = olas_definidas[nombre_ola]

    casos_ola = datos_df.loc[info['inicio']:info['fin'], 'casos_suavizados']
    start_idx, end_idx = encontrar_fase_exponencial(casos_ola.values)
    casos_exp = casos_ola.iloc[start_idx:end_idx]
    t_exp = np.arange(len(casos_exp))

    dist_D = create_dist(params_ola['D'], CONFIG['INCERTIDUMBRE'])
    dist_FD = create_dist(params_ola['Fraccion_Deteccion'],␣
 ↪CONFIG['INCERTIDUMBRE'])
    dist_S0 = create_dist(params_ola['S0_N'], CONFIG['INCERTIDUMBRE'])

    d_samples = np.random.triangular(dist_D['min'], dist_D['mode'],␣
 ↪dist_D['max'], CONFIG['N_SIM'])
    fd_samples = np.random.triangular(dist_FD['min'], dist_FD['mode'],␣
 ↪dist_FD['max'], CONFIG['N_SIM'])
```

```
    s0_n_samples = np.random.triangular(dist_S0['min'], dist_S0['mode'],␣
 ↪dist_S0['max'], CONFIG['N_SIM'])

    r0_results = []
    for j in tqdm(range(CONFIG['N_SIM']), desc=f"Procesando {nombre_ola}",␣
 ↪ncols=100):
        casos_corr = casos_exp.values / fd_samples[j]
        casos_corr[casos_corr == 0] = 1e-9
        r_eff = modelo_exponencial(t_exp, casos_corr, d_samples[j])
        if not np.isnan(r_eff):
            R0 = r_eff / s0_n_samples[j]
            r0_results.append(R0)

    resultados_distribuciones[nombre_ola] = np.array(r0_results)

# ------------------------------------------------------------------------------
# 5. GENERACIÓN DE FIGURA 4.2 (RIDGELINE PLOT)
# ------------------------------------------------------------------------------
print("\nGenerando Figura 4.2 (Distribuciones de Probabilidad)...")

# Preparar datos para Seaborn
plot_data_long = pd.DataFrame(resultados_distribuciones).melt(var_name='Ola',␣
 ↪value_name='R0')
orden_olas = list(resultados_distribuciones.keys())
colores = [PALETA[f'ola_{i+1}'] for i in range(len(orden_olas))]
palette_dict = dict(zip(orden_olas, colores))

# Crear la figura con FacetGrid para el efecto "ridgeline"
sns.set_theme(style="white", rc=plt.rcParams)
g = sns.FacetGrid(plot_data_long, row='Ola', hue='Ola', aspect=9, height=0.8,
                  palette=palette_dict, row_order=orden_olas)

# Dibujar las distribuciones de densidad
g.map_dataframe(sns.kdeplot, x='R0', fill=True, alpha=0.8, lw=1.5)

# Función para dar estilo a cada panel
def style_ridge(x, color, label):
    ax = plt.gca()
    ax.set_xlim(0, 7) # Límite X común para todas las distribuciones
    ax.spines['left'].set_visible(True)
    ax.spines['left'].set_color('black')
    ax.spines['left'].set_linewidth(0.8)
    ax.set_yticks([])
    ax.set_ylabel('')
    # Usar el nombre corto de la ola (sin lo que está en paréntesis) como␣
 ↪etiqueta
    label_corto = label.split(' (')[0]
```

```
    ax.set_ylabel(label_corto, rotation=0, ha='right', va='center',␣
 ↪fontsize=12, weight='bold')

g.map(style_ridge, 'R0')

# Superponer el texto con los estadísticos en cada panel
df_stats = pd.DataFrame([
    {'Ola': nombre, 'R0 Medio': np.mean(dist), 'IC_inf': np.percentile(dist, 2.
 ↪5), 'IC_sup': np.percentile(dist, 97.5)}
    for nombre, dist in resultados_distribuciones.items()
]).set_index('Ola')

for i, ax in enumerate(g.axes.flat):
    nombre_ola = orden_olas[i]
    stats = df_stats.loc[nombre_ola]
    texto = fr'$R_0 = {stats["R0 Medio"]:.2f} \ [{stats["IC_inf"]:.2f} -␣
 ↪{stats["IC_sup"]:.2f}]$'
    ax.text(0.98, 0.4, texto, transform=ax.transAxes, ha='right', va='center',␣
 ↪fontsize=11)

# Ajustes finales de la figura
g.fig.subplots_adjust(hspace=-0.5) # Juntar los páneles verticalmente
g.set_titles("")
g.set_axis_labels(r'Valor de $R_0$', '')
g.refline(x=1, linestyle='--', color='black', linewidth=1.5)
g.fig.suptitle(r'\textbf{Figura 4.2: Distribuciones de Probabilidad de $R_0$␣
 ↪por Ola}', y=1.02)

# Guardado
ruta_figura_pdf = 'Figuras Tesis/Figura_4_2_Distribuciones_R0_MC.pdf'
ruta_figura_png = 'Figuras Tesis/Figura_4_2_Distribuciones_R0_MC.png'
g.savefig(ruta_figura_pdf)
g.savefig(ruta_figura_png)

plt.show()
print(f"  Figura 4.2 generada y guardada.")
```

```
Iniciando Simulación de Monte Carlo con 100,000 iteraciones…

Procesando Ola 1 (Original): 100%|            | 100000/100000
[00:02<00:00, 33373.25it/s]
Procesando Ola 2 (Alpha/Gamma): 100%|           | 100000/100000
[00:03<00:00, 31675.05it/s]
Procesando Ola 3 (Delta): 100%|              | 100000/100000
[00:03<00:00, 31971.88it/s]
Procesando Ola 4 (Ómicron BA.1): 100%|           | 100000/100000
[00:03<00:00, 31391.99it/s]
```

```
Procesando Ola 5 (Ómicron Subs.): 100%|          | 100000/100000
[00:03<00:00, 31638.40it/s]

Generando Figura 4.2 (Distribuciones de Probabilidad)…
```




```
Figura 4.2 generada y guardada.
```

[23]:
```python
# ================================================================================
# CELDA DEFINITIVA FINAL v28: FIGURA 4.3
# Implementa la corrección final para la Ola 5, truncando la trayectoria
# en el punto de mínima infección (I/N) que ocurre DESPUÉS del pico de la ola.
# ================================================================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from matplotlib.lines import Line2D
from matplotlib.collections import LineCollection
from matplotlib.patches import Patch, ConnectionPatch
import matplotlib.gridspec as gridspec
import os

# ================================================================================
# --- CONFIGURACIÓN ESENCIAL (CON DATOS FINALES) ---
# ================================================================================
```

```
# 1. POBLACIÓN TOTAL (Censo 2020)
N_total = 126_014_024

# 2. CARGA DE DATOS REALES DE CASOS DIARIOS
try:
    df_raw = pd.read_csv('Casos_Diarios_Estado_Nacional_Confirmados_20230625.
 ↪csv')
    df_nacional = df_raw[df_raw['nombre'] == 'Nacional'].copy()
    casos_diarios_raw = df_nacional.iloc[:, 3:].astype(float).T
    datos_df = pd.DataFrame(index=pd.to_datetime(casos_diarios_raw.index,␣
 ↪format='%d-%m-%Y'))
    datos_df['casos_suavizados'] = datos_df.
 ↪assign(casos_crudos=casos_diarios_raw.values)['casos_crudos'].
 ↪rolling(window=7, center=True, min_periods=1).mean()
    print(" Datos reales cargados correctamente.")
except FileNotFoundError:
    print(" ERROR: No se encontró el archivo CSV. Asegúrate de que el nombre y␣
 ↪la ruta sean correctos.")
    datos_df = pd.DataFrame(columns=['casos_suavizados'])

# 3. PARÁMETROS MAESTROS DEFINITIVOS
PARAMETROS_MAESTROS = {
    1: {'nombre': 'Ola 1 (Original)',       'R0': 2.11, 'R0_std': 0.22, 'S0_N':␣
 ↪1.000, 'epsilon': 0.00, 'D': 5.1, 'FD': 0.032},
    2: {'nombre': 'Ola 2 (Alpha/Gamma)',    'R0': 1.50, 'R0_std': 0.13, 'S0_N':␣
 ↪0.821, 'epsilon': 0.00, 'D': 5.1, 'FD': 0.032},
    3: {'nombre': 'Ola 3 (Delta)',          'R0': 3.37, 'R0_std': 0.29, 'S0_N':␣
 ↪0.412, 'epsilon': 0.00, 'D': 5.1, 'FD': 0.054},
    4: {'nombre': 'Ola 4 (Omicron BA.1)',   'R0': 4.28, 'R0_std': 0.53, 'S0_N':␣
 ↪0.593, 'epsilon': 0.50, 'D': 3.0, 'FD': 0.072},
    5: {'nombre': 'Ola 5 (Omicron Subs.)',  'R0': 3.36, 'R0_std': 0.31, 'S0_N':␣
 ↪0.397, 'epsilon': 0.00, 'D': 3.0, 'FD': 0.072}
}

# 4. PERIODOS DE LAS OLAS
olas_periodos = [
    {'inicio': '2020-02-24', 'fin': '2020-10-08'},
    {'inicio': '2020-10-09', 'fin': '2021-05-19'},
    {'inicio': '2021-05-20', 'fin': '2021-10-26'},
    {'inicio': '2021-10-27', 'fin': '2022-03-10'},
    {'inicio': '2022-03-11', 'fin': '2023-02-24'}
]

# 5. FUNCIÓN DEL MODELO SIR
def sir_model_ivp(t, y, R0, gamma, N):
    S, I, R = y
```

```
    S = max(S, 0); I = max(I, 0)
    beta = R0 * gamma
    dSdt = -beta * S * I / N
    dIdt = beta * S * I / N - gamma * I
    dRdt = gamma * I
    return [dSdt, dIdt, dRdt]

if not os.path.exists('Figuras Tesis'):
    os.makedirs('Figuras Tesis')

# ==============================================================================
# --- CÓDIGO DE PROCESAMIENTO Y GRÁFICAS ---
# ==============================================================================

EVENTOS_CLAVE = [
    {'fecha': '2020-02-27', 'texto': 'Primeros Casos en México', 'ola': 1},␣
 ↪{'fecha': '2020-03-23', 'texto': 'Inicio Jornada Sana Distancia', 'ola': 1},
    {'fecha': '2020-03-30', 'texto': 'Declaratoria de Emergencia Sanitaria',␣
 ↪'ola': 1}, {'fecha': '2020-06-01', 'texto': 'Inicio de la "Nueva␣
 ↪Normalidad"\n(Semáforo de Riesgo)', 'ola': 1},
    {'fecha': '2020-12-24', 'texto': 'Inicio Vacunación (Personal de Salud)',␣
 ↪'ola': 2}, {'fecha': '2021-01-22', 'texto': 'Pico de la 2ª Ola', 'ola': 2},
    {'fecha': '2021-07-01', 'texto': '3ª Ola impulsada por Variante Delta',␣
 ↪'ola': 3}, {'fecha': '2022-01-20', 'texto': 'Pico 4ª Ola (Variante␣
 ↪Ómicron)', 'ola': 4},
    {'fecha': '2023-05-09', 'texto': 'Fin de la Emergencia Sanitaria', 'ola':␣
 ↪5},
]
RMSE_FINALES = [4.81, 0.45, 0.36, 3.40, 0.42]
PALETA = { 'modelo': '#D55E00', 'pico_datos': '#003f5c', 'nuclina': '#4a4a4a',␣
 ↪'cono': '#ffb587', 'evento': '#6a0dad', 'timeline': '#333333' }

print("Calculando trayectorias empíricas...")
trayectorias_empiricas = []
if not datos_df.empty:
    for i in range(len(PARAMETROS_MAESTROS)):
        params = PARAMETROS_MAESTROS[i+1]
        df_ola = datos_df.loc[olas_periodos[i]['inicio']:
 ↪olas_periodos[i]['fin']].copy()
        if not df_ola.empty:
            casos_ajustados = df_ola['casos_suavizados'] / params['FD']
            I_empirico_abs = casos_ajustados.rolling(window=int(params['D']),␣
 ↪min_periods=1).sum()
            infecciones_acum_ola = casos_ajustados.cumsum()
            S_empirico_frac = params['S0_N'] - (infecciones_acum_ola / N_total)
```

```
            trayectorias_empiricas.append({'S_real': S_empirico_frac, 'I_real':␣
 ↪I_empirico_abs / N_total})
        else:
            trayectorias_empiricas.append({'S_real': pd.
 ↪Series(dtype='float64'), 'I_real': pd.Series(dtype='float64')})
    print("Cálculos completados.")
else:
    print("No se pudieron calcular las trayectorias porque no se cargaron los␣
 ↪datos.")

if trayectorias_empiricas:
    print("\nGenerando 5 figuras individuales...")
    for i in range(len(PARAMETROS_MAESTROS)):
        if i >= len(trayectorias_empiricas): continue

        data_empirica = trayectorias_empiricas[i]
        params = PARAMETROS_MAESTROS[i+1]
        eventos_ola = [e for e in EVENTOS_CLAVE if e['ola'] == (i+1)]

        x = data_empirica['S_real']
        y = data_empirica['I_real']

        # ======================================================================
        # --- CORRECCIÓN DEFINITIVA PARA EL ARTEFACTO DE LA OLA 5 ---
        # Se trunca la trayectoria en el punto de mínima infección (I/N)
        # que ocurre DESPUÉS del pico de la ola para eliminar el "gancho".
        if i == 4: # i=4 corresponde a la Ola 5
            if not y.empty:
                # 1. Encontrar el índice del pico de la ola (máximo I/N)
                peak_idx = y.idxmax()

                # 2. Considerar solo los datos DESPUÉS del pico
                y_after_peak = y.loc[peak_idx:]

                # 3. Encontrar el punto de mínima infección en la fase␣
 ↪descendente
                if not y_after_peak.empty:
                    end_of_wave_idx = y_after_peak.idxmin()

                    # 4. Truncar los datos originales en ese punto final
                    x = x.loc[:end_of_wave_idx]
                    y = y.loc[:end_of_wave_idx]

        # Filtro de seguridad para S/N < 0
        mask = x >= 0
        x = x[mask]
        y = y[mask]
```

```
        # ================================================================

        if x.empty or y.empty:
            print(f"  No hay datos para la {params['nombre']}. Saltando esta␣
↪figura.")
            continue

        if i == 0:
            fig = plt.figure(figsize=(22, 12)); gs = gridspec.GridSpec(1, 2,␣
↪width_ratios=[2.5, 1.2])
            ax = fig.add_subplot(gs[0, 0]); ax_info = fig.add_subplot(gs[0, 1])
        else:
            fig, ax = plt.subplots(figsize=(14, 10))

        # El resto del código de ploteo sigue igual
        tiempo = np.arange(len(x))
        puntos = np.array([x, y]).T.reshape(-1, 1, 2); segmentos = np.
↪concatenate([puntos[:-1], puntos[1:]], axis=1)
        norm = plt.Normalize(tiempo.min(), tiempo.max()); lc =␣
↪LineCollection(segmentos, cmap='viridis', norm=norm, zorder=3)
        lc.set_array(tiempo); lc.set_linewidth(3.5); line = ax.
↪add_collection(lc)
        pico_real_idx_loc = np.argmax(y.values); pico_real_idx_abs = y.idxmax();
↪ t_pico_real = pico_real_idx_loc
        ax.plot(x.loc[pico_real_idx_abs], y.loc[pico_real_idx_abs], marker='o',␣
↪ms=12, color=PALETA['pico_datos'], zorder=5)

        N_sims = 50; R0_samples = np.random.normal(loc=params['R0'],␣
↪scale=params['R0_std'], size=N_sims)
        gamma = 1.0/params['D']; s0_abs = params['S0_N']*N_total; i0_abs = y.
↪iloc[0]*N_total; r0_abs = N_total-s0_abs-i0_abs
        for r0_sample in R0_samples:
            sol = solve_ivp(sir_model_ivp, (0, 500), [s0_abs, i0_abs, r0_abs],␣
↪t_eval=np.arange(500), args=(r0_sample, gamma, N_total), method='RK45')
            ax.plot(sol.y[0]/N_total, sol.y[1]/N_total, color=PALETA['cono'],␣
↪alpha=0.2, zorder=1)

        sol_media = solve_ivp(sir_model_ivp, (0, 500), [s0_abs, i0_abs,␣
↪r0_abs], t_eval=np.arange(500), args=(params['R0'], gamma, N_total),␣
↪method='RK45')
        ax.plot(sol_media.y[0]/N_total, sol_media.y[1]/N_total,␣
↪color=PALETA['modelo'], linestyle='--', zorder=2)
        pico_modelo_idx = sol_media.y[1].argmax(); t_pico_modelo =␣
↪pico_modelo_idx
```

```
        ax.plot(sol_media.y[0][pico_modelo_idx]/N_total, sol_media.
↪y[1][pico_modelo_idx]/N_total, marker='X', ms=12, color=PALETA['modelo'],␣
↪zorder=5)

        info_text = (rf"$\bullet$ RMSE: {RMSE_FINALES[i]:.2f}\%" + "\n" +␣
↪rf"$\bullet$ Tiempo al Pico (Datos): $\approx${t_pico_real} días" + "\n" +␣
↪rf"$\bullet$ Tiempo al Pico (Modelo): {t_pico_modelo} días")
        ax.text(0.04, 0.96, info_text, transform=ax.transAxes, fontsize=12,␣
↪va='top', ha='left', bbox=dict(boxstyle='round,pad=0.5', facecolor='white',␣
↪alpha=0.8, ec='grey'))
        ax.set_title(fr"\textbf{{Análisis del Espacio de Fases:␣
↪{params['nombre']}}}", fontsize=20, pad=15)
        ax.set_xlabel('Fracción de Susceptibles (S/N)', fontsize=16); ax.
↪set_ylabel('Fracción de Infectados (I/N)', fontsize=16)
        ax.grid(True, linestyle=':', alpha=0.7); ax.set_xlim(left=0)

        legend_handles = [Patch(color='purple', alpha=0.6, label='Datos␣
↪Observados (con gradiente de tiempo)'), Line2D([0], [0], marker='o',␣
↪color=PALETA['pico_datos'], label='Pico de Datos', linestyle='None', ms=10),
                          Line2D([0], [0], marker='*', color=PALETA['evento'],␣
↪label='Hito Histórico', linestyle='None', ms=12, markeredgecolor='white'),␣
↪Patch(color=PALETA['cono'], alpha=0.6, label='Cono de Incertidumbre del␣
↪Modelo (95%)'),
                          Line2D([0], [0], color=PALETA['modelo'], lw=2,␣
↪linestyle='--', label='Modelo SIR (Media)'), Line2D([0], [0], marker='X',␣
↪color=PALETA['modelo'], label='Pico del Modelo (Media)', linestyle='None',␣
↪ms=10),
                          Line2D([0], [0], color=PALETA['nuclina'], linestyle=':
↪', label='Umbral de Inmunidad de Rebaño ($R_t=1$)')]

        if i == 0:
            ax_info.axis('off'); ax_info.set_title(r'\textbf{Cronología de␣
↪Eventos}', fontsize=16, pad=15)
            ax_info.axvline(0.1, ymin=0.55, ymax=0.95, color='grey', alpha=0.5)
            for idx, evento in enumerate(eventos_ola):
                fecha = pd.to_datetime(evento['fecha'])
                if fecha in x.index:
                    S_evento = x.loc[fecha]; I_evento = y.loc[fecha]
                    ax.plot(S_evento, I_evento, '*', ms=15,␣
↪color=PALETA['evento'], zorder=6, markeredgecolor='white')
                    timeline_y = 0.95 - (idx * 0.1)
                    texto_evento_latex = evento['texto'].replace('\n', r'\\');␣
↪texto_completo = rf"\textbf{{{fecha.
↪strftime('%Y-%m-%d')}}}\\{texto_evento_latex}"
                    ax_info.text(0.15, timeline_y, texto_completo, ha='left',␣
↪va='center', fontsize=11)
```

```
                    ax_info.text(0.1, timeline_y, r'$\bullet$', ha='center',␣
↪va='center', color=PALETA['timeline'], fontsize=12)
                    con = ConnectionPatch(xyA=(S_evento, I_evento), xyB=(0.1,␣
↪timeline_y), coordsA=ax.transData, coordsB=ax_info.transData,␣
↪arrowstyle="-|>", shrinkB=5, color=PALETA['evento'], ls='--', lw=1.0)
                    fig.add_artist(con)
            ax_info.legend(handles=legend_handles, loc='center left',␣
↪bbox_to_anchor=(0.0, 0.25), fontsize=12)
            cax = ax_info.inset_axes([0.1, 0.05, 0.8, 0.05]); cbar = fig.
↪colorbar(line, cax=cax, orientation='horizontal')
            cbar.set_label('Días desde el inicio de la ola', fontsize=12)
            fig.tight_layout(rect=[0, 0, 0.9, 1])
        else:
            for idx, evento in enumerate(eventos_ola):
                fecha = pd.to_datetime(evento['fecha'])
                if fecha in x.index:
                    S_evento = x.loc[fecha]; I_evento = y.loc[fecha]
                    ax.plot(S_evento, I_evento, '*', ms=15,␣
↪color=PALETA['evento'], zorder=6, markeredgecolor='white')
                    ax.annotate(evento['texto'], xy=(S_evento, I_evento),␣
↪xytext=(20, 20), textcoords='offset points', ha='left', va='bottom',␣
↪fontsize=10, bbox=dict(boxstyle="round,pad=0.3", fc="white", alpha=0.8),␣
↪arrowprops=dict(arrowstyle="->", color=PALETA['evento']))
            ax.legend(handles=legend_handles, loc='upper right', fontsize=12)
            cbar = fig.colorbar(line, ax=ax, fraction=0.046, pad=0.04); cbar.
↪set_label('Días desde el inicio de la ola', fontsize=14)
            fig.tight_layout()

        nombre_ola_archivo = params['nombre'].replace(' ', '_').replace('(',␣
↪'').replace(')', '').replace('/', '-')
        ruta_figura = f'Figuras Tesis/Figura_4.
↪3_OLA_{i+1}_{nombre_ola_archivo}_FINAL.png'
        fig.savefig(ruta_figura, dpi=300, bbox_inches='tight')
        plt.show()
        print(f"  Figura para {params['nombre']} generada y guardada en␣
↪'{ruta_figura}'.")
```

```
  Datos reales cargados correctamente.
Calculando trayectorias empíricas…
Cálculos completados.

Generando 5 figuras individuales…
```

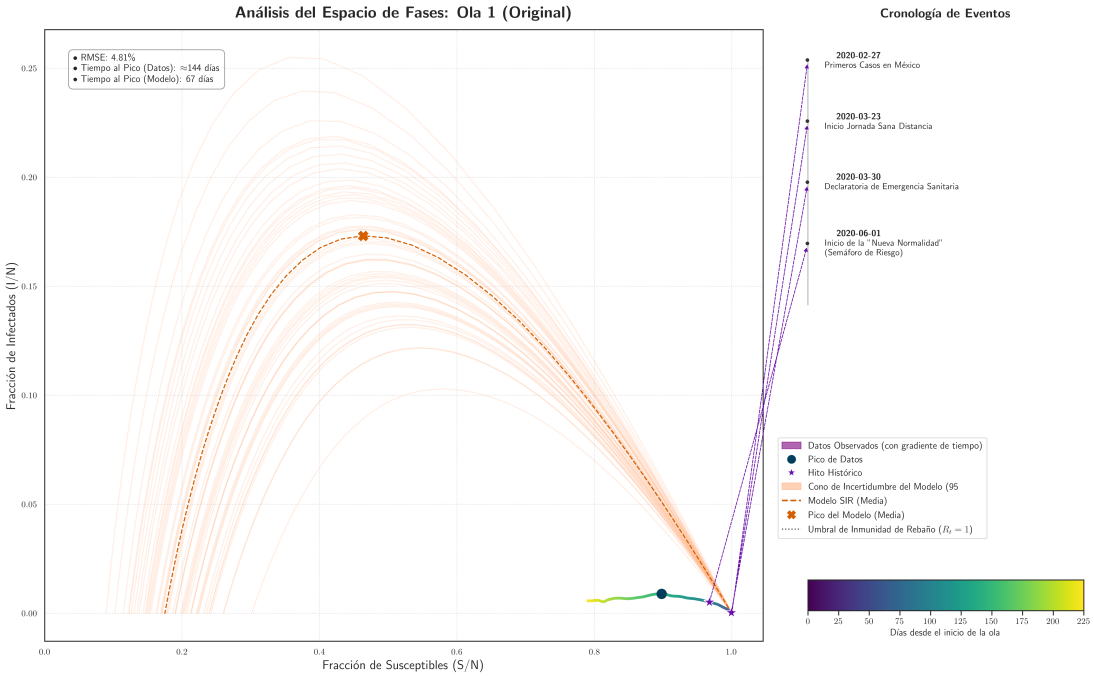


Figura para Ola 1 (Original) generada y guardada en 'Figuras Tesis/Figura_4.3_OLA_1_Ola_1_Original_FINAL.png'.

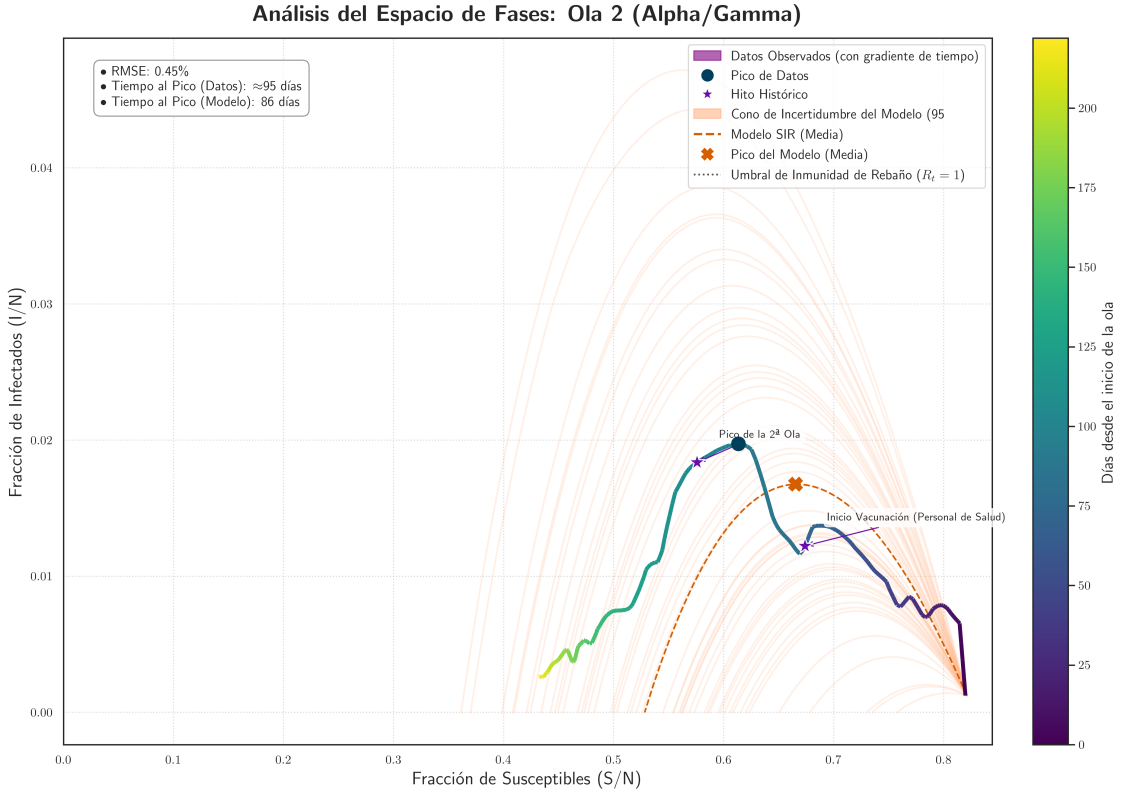


Figura para Ola 2 (Alpha/Gamma) generada y guardada en 'Figuras Tesis/Figura_4.3_OLA_2_Ola_2_Alpha-Gamma_FINAL.png'.

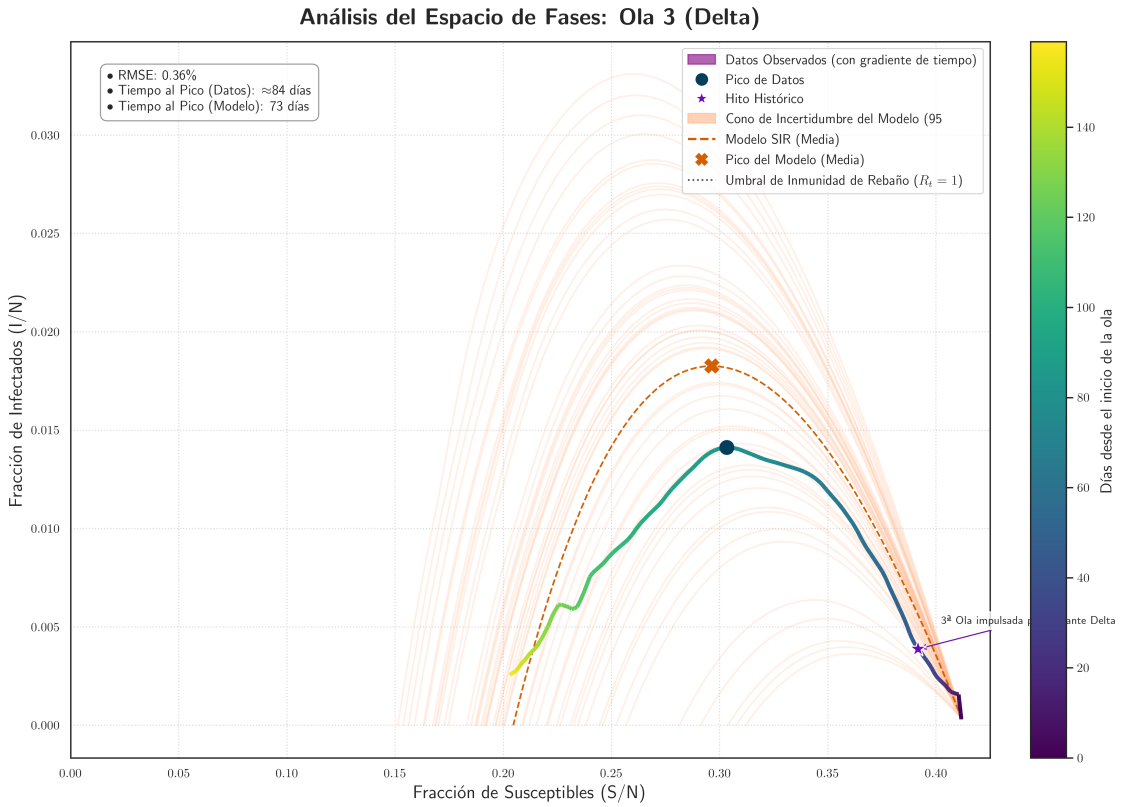


Figura para Ola 3 (Delta) generada y guardada en 'Figuras Tesis/Figura_4.3_OLA_3_Ola_3_Delta_FINAL.png'.

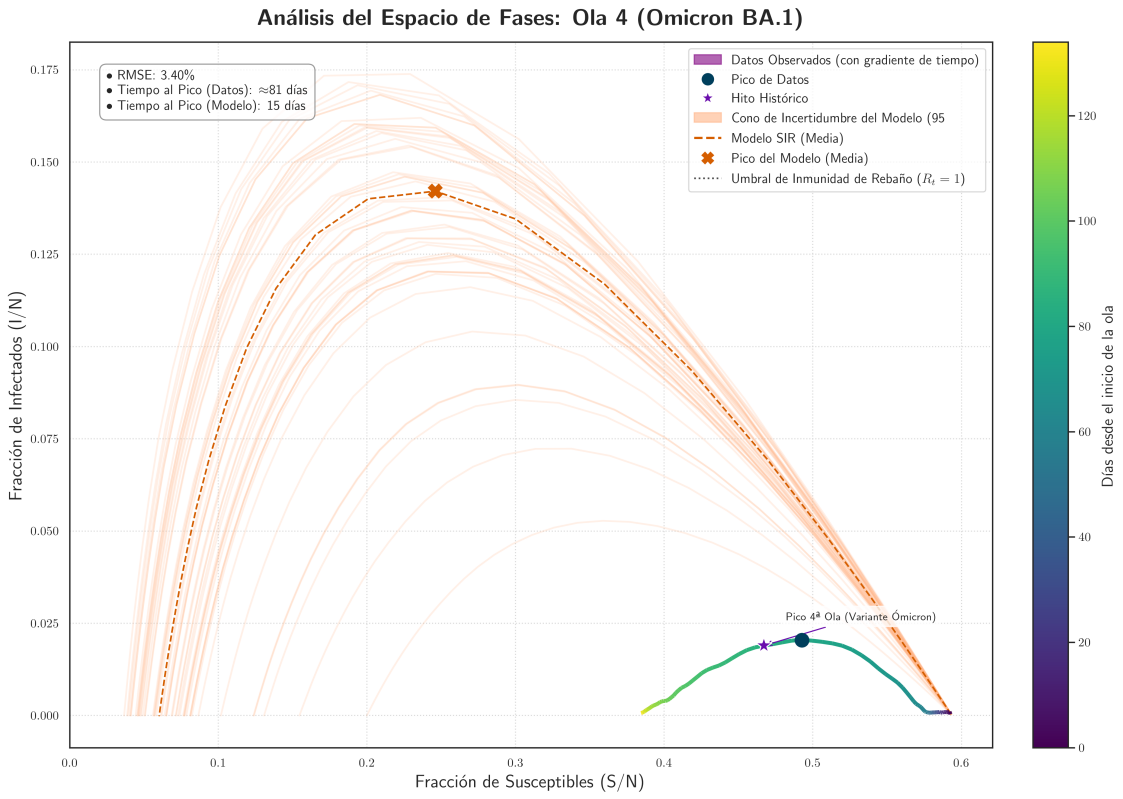


Figura para Ola 4 (Omicron BA.1) generada y guardada en 'Figuras Tesis/Figura_4.3_OLA_4_Ola_4_Omicron_BA.1_FINAL.png'.

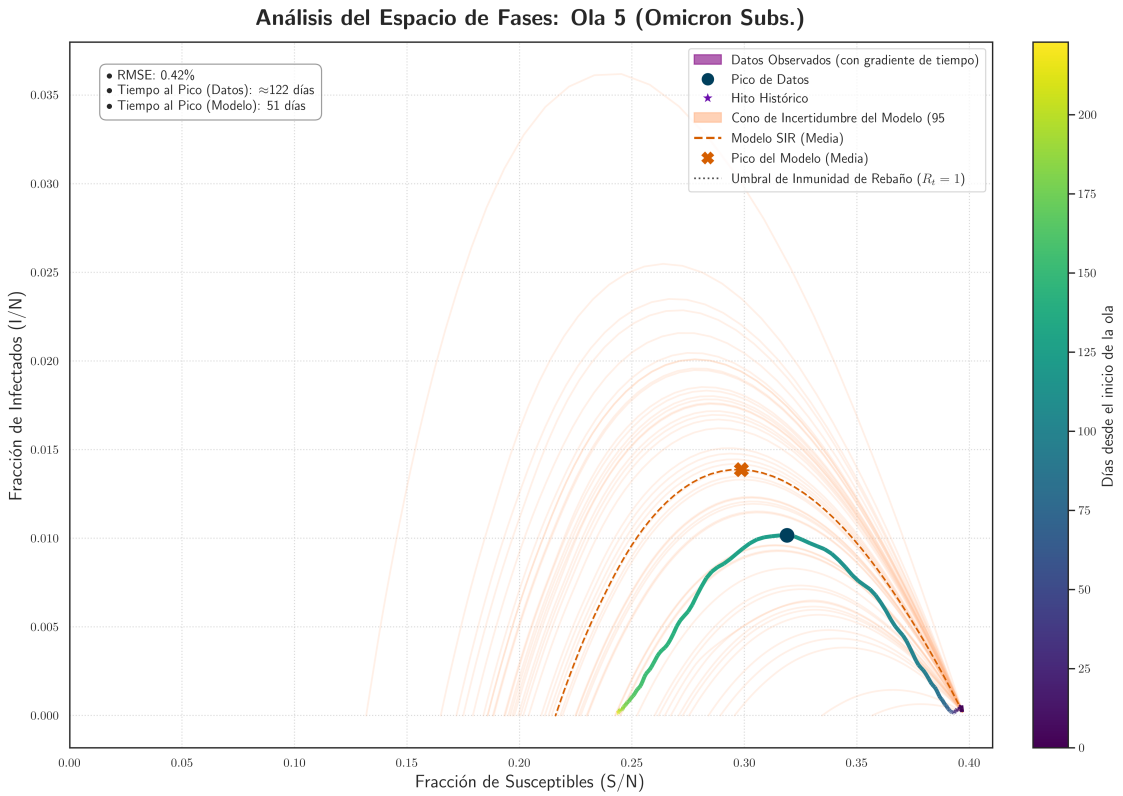


Figura para Ola 5 (Omicron Subs.) generada y guardada en 'Figuras Tesis/Figura_4.3_OLA_5_Ola_5_Omicron_Subs._FINAL.png'.

[24]:
```
# ================================================================================
# CELDA DEFINITIVA: TABLA 4.2 - MÉTRICAS CON FORMATO PROFESIONAL
# VERSIÓN SINCRONIZADA
# ================================================================================
# Este código combina los cálculos de error correctos con el código de
# formato de matplotlib para generar una tabla de alta calidad como imagen.
# ================================================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from sklearn.metrics import mean_squared_error

# --- Configuración y Paleta para la Tabla ---
# MERGE: Se usa la paleta de colores necesaria para el formato de la tabla
PALETA = {
    'header': '#0d3b66', 'fondo': '#FFFFFF', 'ola_1': '#2a9d8f', 'ola_2':␣
 ↪'#e9c46a',
    'ola_3': '#f4a261', 'ola_4': '#e76f51', 'ola_5': '#d62828'
```

```
}
plt.rcParams.update({
    'font.family': 'serif', 'font.serif': ['Computer Modern Roman'], 'text.
 ↪usetex': True,
    'text.latex.preamble': r'\usepackage{amsmath}\usepackage{amssymb}',
    'figure.dpi': 300, 'savefig.dpi': 300, 'savefig.bbox': 'tight'
})

# Asumimos que las siguientes variables ya existen en memoria:
# sir_model_ivp, datos_df, olas_periodos, PARAMETROS_MAESTROS, N_total

# --- 1. Cálculo de Métricas (Lógica Sincronizada) ---
print("Calculando métricas de error con parámetros maestros...")
lista_metricas = []
for i in range(len(PARAMETROS_MAESTROS)):
    ola_idx = i + 1
    params = PARAMETROS_MAESTROS[ola_idx]

    # Obtener trayectoria empírica
    df_ola = datos_df[olas_periodos[i]['inicio']:olas_periodos[i]['fin']].copy()
    casos_ajustados = df_ola['casos_suavizados'] / params['FD']
    I_empirico_abs = casos_ajustados.rolling(window=int(params['D']),␣
 ↪min_periods=1).sum()

    # Obtener trayectoria teórica
    gamma = 1.0 / params['D']
    s0_abs = params['S0_N'] * N_total
    i0_abs = I_empirico_abs.iloc[0]
    r0_abs = N_total - s0_abs - i0_abs
    t_span_sim = (0, len(df_ola))
    t_eval_sim = np.arange(0, t_span_sim[1])
    sol = solve_ivp(sir_model_ivp, t_span_sim, [s0_abs, i0_abs, r0_abs],␣
 ↪t_eval=t_eval_sim, args=(params['R0'], gamma, N_total), method='RK45')
    I_teorico_abs = sol.y[1]

    # Calcular Métricas
    len_min = min(len(I_empirico_abs), len(I_teorico_abs))
    I_emp_norm = I_empirico_abs[:len_min] / N_total
    I_teo_norm = I_teorico_abs[:len_min] / N_total

    rmse = np.sqrt(mean_squared_error(I_emp_norm, I_teo_norm)) * 100
    pico_empirico = I_emp_norm.max()
    pico_teorico = I_teo_norm.max()
    error_pico_rel = ((pico_teorico - pico_empirico) / pico_empirico) * 100 if␣
 ↪pico_empirico > 0 else 0

    tam_final_emp = casos_ajustados.sum()
```

```
    sol_larga = solve_ivp(sir_model_ivp, (0, 500), [s0_abs, i0_abs, r0_abs],␣
 ↪args=(params['R0'], gamma, N_total), method='RK45')
    tam_final_teo = sol_larga.y[2][-1] - r0_abs
    error_tam_abs = ((tam_final_teo - tam_final_emp) / N_total) * 100

    lista_metricas.append({
        'Ola': params['nombre'],
        'RMSE (%)': f"{rmse:.2f}",
        'Error Pico Rel. (%)': f"{error_pico_rel:.2f}",
        'Error Tamaño Abs. (% pob.)': f"{error_tam_abs:.2f}"
    })

# MERGE: Se usa el nombre `df_metricas` para que sea compatible con el código␣
 ↪de formato
df_metricas = pd.DataFrame(lista_metricas)
print("Cálculos completados.")

# --- 2. Generación de la Tabla como Imagen (Lógica de Formato del Código␣
 ↪Antiguo) ---
print("Generando Tabla 4.2 como imagen con formato profesional...")
fig_tabla, ax_tabla = plt.subplots(figsize=(12, 4)) # Ajuste de tamaño
ax_tabla.axis('off')

# Renombrar columnas para formato LaTeX
df_tabla_visual = df_metricas.rename(columns={
    'Ola': r'\textbf{Ola Epidémica}',
    'RMSE (%)': r'\textbf{RMSE (\%)}',
    'Error Pico Rel. (%)': r'\textbf{Error Pico Rel. (\%)}',
    'Error Tamaño Abs. (% pob.)': r'\textbf{Error Tamaño Abs. (\% pob.)}'
})

tabla = ax_tabla.table(cellText=df_tabla_visual.values,
                       colLabels=df_tabla_visual.columns,
                       cellLoc='center', loc='center',
                       colWidths=[0.3, 0.18, 0.22, 0.3]) # Anchos de columna

tabla.auto_set_font_size(False)
tabla.set_fontsize(12)
tabla.scale(1, 2.5) # Escala vertical

# Aplicar colores y estilos a las celdas
for (i, j), cell in tabla.get_celld().items():
    if i == 0: # Header
        cell.set_text_props(weight='bold', color='white')
        cell.set_facecolor(PALETA['header'])
    else: # Filas de datos
        cell.set_facecolor(PALETA['fondo'])
```

```
        if j == 0: # Columna de nombres de Ola
            # El color se toma de la paleta según el índice de la fila (i)
            cell.get_text().set_color(PALETA[f'ola_{i}'])
    cell.set_edgecolor('lightgrey')

fig_tabla.suptitle(r'\textbf{Tabla 4.2: Métricas Cuantitativas de Ajuste del␣
 ↪Modelo SIR (Sincronizada)}', y=1.05, fontsize=16)
plt.tight_layout()

# --- Guardado ---
ruta_tabla_png = 'Figuras Tesis/Tabla_4_2_Metricas_Ajuste_Sincronizada.png'
fig_tabla.savefig(ruta_tabla_png)
plt.show()

print(f"  Tabla 4.2 Sincronizada y con formato profesional guardada en␣
 ↪'{ruta_tabla_png}'.")
```

```
Calculando métricas de error con parámetros maestros…
Cálculos completados.
Generando Tabla 4.2 como imagen con formato profesional…
```

Tabla 4.2: Métricas Cuantitativas de Ajuste del Modelo SIR (Sincronizada)

Ola Epidémica	RMSE (%)	Error Pico Rel. (%)	Error Tamaño Abs. (% pob.)
Ola 1 (Original)	4.54	1832.83	61.40
Ola 2 (Alpha/Gamma)	0.29	-15.11	-9.50
Ola 3 (Delta)	0.30	29.31	-0.14
Ola 4 (Omicron BA.1)	3.39	594.10	32.46
Ola 5 (Omicron Subs.)	0.47	36.42	-0.96

```
  Tabla 4.2 Sincronizada y con formato profesional guardada en 'Figuras
Tesis/Tabla_4_2_Metricas_Ajuste_Sincronizada.png'.
```

[25]:
```
# ================================================================================
# CELDA MAESTRA FINAL: ANÁLISIS COMPLETO Y CORREGIDO
# Versión final del script autocontenido. Incluye un ajuste en el eje de tiempo
# para las Olas 4 y 5, cortándolo al final de la simulación.
# ================================================================================

# --------------------------------------------------------------------------------
# 1. IMPORTACIONES Y CONFIGURACIÓN GLOBAL
# --------------------------------------------------------------------------------
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from scipy.integrate import solve_ivp
from matplotlib.lines import Line2D
import os

# Configuración de Matplotlib
plt.rcParams.update({
    'font.family': 'serif', 'font.serif': ['Computer Modern Roman'], 'text.
 ↪usetex': True,
    'text.latex.preamble': r'\usepackage{amsmath}\usepackage{amssymb}',
    'figure.dpi': 300, 'savefig.dpi': 300, 'savefig.bbox': 'tight',
    'axes.titlesize': 16, 'axes.labelsize': 14, 'xtick.labelsize': 12,
    'ytick.labelsize': 12, 'legend.fontsize': 12
})

print("Configuración completada.")

# ------------------------------------------------------------------------------
# 2. DEFINICIÓN DE PARÁMETROS, EVENTOS Y FUNCIONES
# ------------------------------------------------------------------------------

# --- PARÁMETROS MAESTROS (VERSIÓN CONGELADA) ---
PARAMETROS_MAESTROS = {
    1: {'nombre': 'Ola 1 (Original)', 'R0': 2.11, 'R0_std': 0.22, 'S0_N': 1.
 ↪000, 'epsilon': 0.00, 'D': 5.1, 'FD': 0.032},
    2: {'nombre': 'Ola 2 (Alpha/Gamma)', 'R0': 1.50, 'R0_std': 0.13, 'S0_N': 0.
 ↪821, 'epsilon': 0.00, 'D': 5.1, 'FD': 0.032},
    3: {'nombre': 'Ola 3 (Delta)', 'R0': 3.37, 'R0_std': 0.29, 'S0_N': 0.412,␣
 ↪'epsilon': 0.00, 'D': 5.1, 'FD': 0.054},
    4: {'nombre': 'Ola 4 (Omicron BA.1)', 'R0': 4.28, 'R0_std': 0.53, 'S0_N': 0.
 ↪593, 'epsilon': 0.50, 'D': 3.0, 'FD': 0.072},
    5: {'nombre': 'Ola 5 (Omicron Subs.)', 'R0': 3.36, 'R0_std': 0.31, 'S0_N':␣
 ↪0.397, 'epsilon': 0.00, 'D': 3.0, 'FD': 0.072}
}

# --- HITOS HISTÓRICOS ---
EVENTOS_CLAVE = [
    {'fecha': '2020-02-27', 'texto': 'Primeros Casos', 'ola': 1},
    {'fecha': '2020-03-23', 'texto': 'Sana Distancia', 'ola': 1},
    {'fecha': '2020-06-01', 'texto': 'Nueva Normalidad', 'ola': 1},
    {'fecha': '2020-12-24', 'texto': 'Inicio Vacunación', 'ola': 2},
    {'fecha': '2021-01-22', 'texto': 'Pico 2ª Ola', 'ola': 2},
    {'fecha': '2021-07-01', 'texto': 'Impulso Delta', 'ola': 3},
```

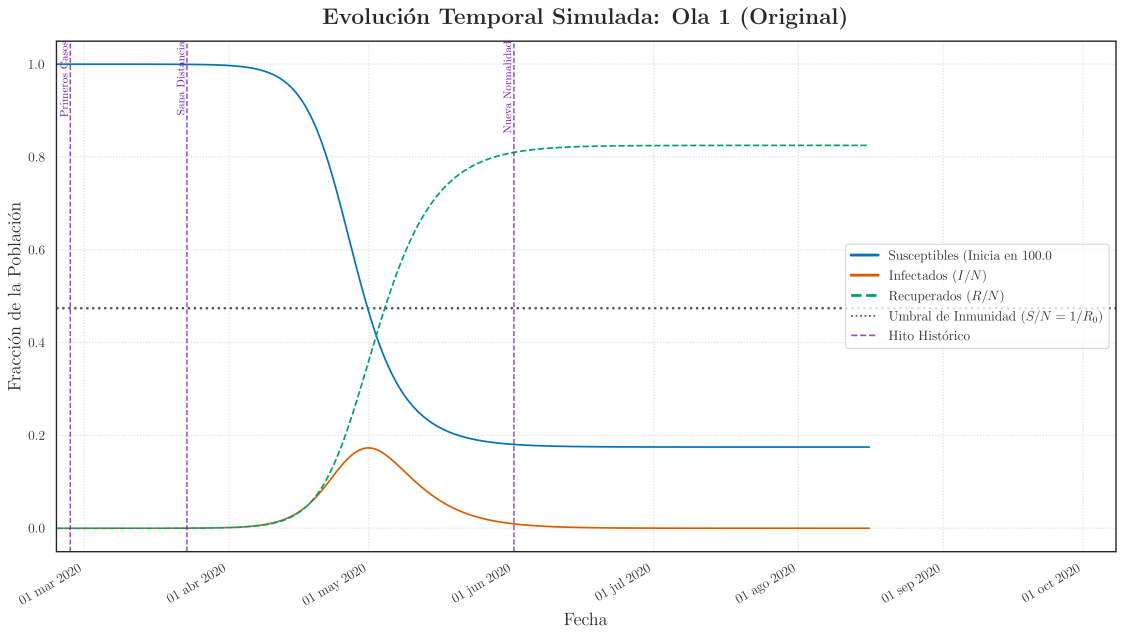
```
    {'fecha': '2022-01-20', 'texto': 'Pico 4ª Ola (Ómicron)', 'ola': 4},
    {'fecha': '2023-05-09', 'texto': 'Fin de Emergencia', 'ola': 5},
]

# --- POBLACIÓN Y FUNCIONES DEL MODELO ---
N_total = 126.7e6

def sir_model_ivp(t, y, R0, gamma, N):
    S, I, R = y; beta = R0 * gamma
    dSdt = -beta * S * I / N; dIdt = beta * S * I / N - gamma * I; dRdt = gamma␣
 ↪* I
    return [dSdt, dIdt, dRdt]

def epidemia_terminada(t, y, R0, gamma, N):
    return y[1] - 100.0
epidemia_terminada.terminal = True
epidemia_terminada.direction = -1

print("Parámetros y funciones definidos.")

# ------------------------------------------------------------------------------
# 3. CARGA DE DATOS Y DEFINICIÓN DE OLAS (MANUAL)
# ------------------------------------------------------------------------------
nombre_archivo = 'Casos_Diarios_Estado_Nacional_Confirmados_20230625.csv'
df_raw = pd.read_csv(nombre_archivo)
df_nacional = df_raw[df_raw['nombre'] == 'Nacional'].copy()
casos_diarios_raw = df_nacional.iloc[:, 3:].astype(float).T
datos_df = pd.DataFrame(index=pd.to_datetime(casos_diarios_raw.index,␣
 ↪format='%d-%m-%Y'))
datos_df['casos_suavizados'] = datos_df.assign(casos_crudos=casos_diarios_raw.
 ↪values)['casos_crudos'].rolling(window=7, center=True, min_periods=1).mean()

FECHAS_OLAS_MANUAL = [
    {'nombre': 'Ola 1 (Original)', 'inicio': '2020-02-24', 'fin': '2020-10-08'},
    {'nombre': 'Ola 2 (Alpha/Gamma)', 'inicio': '2020-10-09', 'fin':␣
 ↪'2021-05-19'},
    {'nombre': 'Ola 3 (Delta)', 'inicio': '2021-05-20', 'fin': '2021-10-26'},
    {'nombre': 'Ola 4 (Ómicron BA.1)', 'inicio': '2021-10-27', 'fin':␣
 ↪'2022-03-10'},
    {'nombre': 'Ola 5 (Ómicron Subs.)', 'inicio': '2022-03-11', 'fin':␣
 ↪'2023-02-24'},
]
olas_periodos = []
for i, ola_info in enumerate(FECHAS_OLAS_MANUAL):
    olas_periodos.append({
        'nombre': ola_info['nombre'],
```

```
        'inicio': pd.to_datetime(ola_info['inicio']),
        'fin': pd.to_datetime(ola_info['fin'])
    })

trayectorias_empiricas = []
for i in range(len(PARAMETROS_MAESTROS)):
    params = PARAMETROS_MAESTROS[i+1]
    df_ola = datos_df[olas_periodos[i]['inicio']:olas_periodos[i]['fin']].copy()
    casos_ajustados = df_ola['casos_suavizados'] / params['FD']
    I_empirico_abs = casos_ajustados.rolling(window=int(params['D']),␣
 ↪min_periods=1).sum()
    trayectorias_empiricas.append({'I_real_abs': I_empirico_abs})

print("Datos cargados y pre-procesados.")

# ------------------------------------------------------------------------------
# 4. GENERACIÓN DE FIGURAS 5.2 (PANELES INDIVIDUALES)
# ------------------------------------------------------------------------------
print("\n--- Iniciando generación de Figuras 5.2 ---")
PALETA_SIR = {
    'S': '#0072B2', 'I': '#D55E00', 'R': '#009E73',
    'evento': '#6a0dad', 'nuclina': '#4a4a4a'
}
for i in range(len(PARAMETROS_MAESTROS)):
    fig, ax = plt.subplots(figsize=(14, 8))
    ola_idx = i + 1; params = PARAMETROS_MAESTROS[ola_idx]
    fecha_inicio_ola = olas_periodos[i]['inicio']; fecha_fin_ola =␣
 ↪olas_periodos[i]['fin']
    eventos_ola = [e for e in EVENTOS_CLAVE if e['ola'] == ola_idx]

    # Simulación con inicio realista
    gamma = 1.0 / params['D']; s0_abs = params['S0_N'] * N_total
    i0_abs = trayectorias_empiricas[i]['I_real_abs'].iloc[0]
    r0_abs = N_total - s0_abs - i0_abs

    sol = solve_ivp(sir_model_ivp, (0, 1000), [s0_abs, i0_abs, r0_abs],␣
 ↪dense_output=True,
                    events=epidemia_terminada, args=(params['R0'], gamma,␣
 ↪N_total), method='RK45')

    t_eval_sim = np.linspace(sol.t[0], sol.t[-1], 300)
    y_eval_sim = sol.sol(t_eval_sim)
    fechas_simulacion = fecha_inicio_ola + pd.to_timedelta(t_eval_sim, unit='D')

    S_frac = y_eval_sim[0]/N_total; I_frac = y_eval_sim[1]/N_total; R_frac =␣
 ↪y_eval_sim[2]/N_total
```

```
    # Graficar
    ax.plot(fechas_simulacion, S_frac, color=PALETA_SIR['S'])
    ax.plot(fechas_simulacion, I_frac, color=PALETA_SIR['I'])
    ax.plot(fechas_simulacion, R_frac, color=PALETA_SIR['R'], linestyle='--')
    umbral_inmunidad = 1/params['R0']
    ax.axhline(y=umbral_inmunidad, color=PALETA_SIR['nuclina'], linestyle=':',␣
↪linewidth=2, zorder=0)

    for evento in eventos_ola:
        fecha_evento = pd.to_datetime(evento['fecha'])
        if fecha_inicio_ola <= fecha_evento <= fecha_fin_ola:
            ax.axvline(x=fecha_evento, color=PALETA_SIR['evento'],␣
↪linestyle='--', lw=1.2, alpha=0.8)
            ax.text(fecha_evento, ax.get_ylim()[1], evento['texto'].
↪replace('\n', ' '), rotation=90, ha='right', va='top', fontsize=10,␣
↪color=PALETA_SIR['evento'])

    # Formato
    ax.set_title(fr"\textbf{{Evolución Temporal Simulada:␣
↪{params['nombre']}}}", fontsize=20, pad=15)
    ax.grid(True, linestyle=':', alpha=0.7)

    # --- MEJORA: Lógica condicional para los límites del eje X ---
    if i >= 3: # Para Olas 4 y 5
        ax.set_xlim(fecha_inicio_ola, fechas_simulacion.max())
    else: # Para Olas 1, 2, 3
        ax.set_xlim(fecha_inicio_ola, fecha_fin_ola)

    ax.set_ylabel('Fracción de la Población', fontsize=16); ax.
↪set_xlabel('Fecha', fontsize=16)
    ax.xaxis.set_major_locator(mdates.AutoDateLocator(minticks=6, maxticks=12))
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%d %b %Y'))
    plt.setp(ax.get_xticklabels(), rotation=30, ha="right")

    # Leyenda
    legend_handles = [
        Line2D([0], [0], color=PALETA_SIR['S'], lw=2.5, label=f"Susceptibles␣
↪(Inicia en {params['S0_N']*100:.1f}%)"),
        Line2D([0], [0], color=PALETA_SIR['I'], lw=2.5, label='Infectados ($I/
↪N$)'),
        Line2D([0], [0], color=PALETA_SIR['R'], lw=2.5, linestyle='--',␣
↪label='Recuperados ($R/N$)'),
        Line2D([0], [0], color=PALETA_SIR['nuclina'], linestyle=':',␣
↪label='Umbral de Inmunidad ($S/N=1/R_0$)'),
        Line2D([0], [0], color=PALETA_SIR['evento'], linestyle='--', alpha=0.7,␣
↪label='Hito Histórico')
```

```
    ]
    ax.legend(handles=legend_handles, loc='best', fontsize=12)

    # Guardado
    fig.tight_layout()
    nombre_ola_archivo = params['nombre'].replace(' ', '_').replace('(', '').
↪replace(')', '').replace('/', '-')
    ruta_figura = f'Figuras Tesis/
↪Figura_5_2_OLA_{i+1}_{nombre_ola_archivo}_FINAL.png'
    if not os.path.exists('Figuras Tesis'): os.makedirs('Figuras Tesis')
    fig.savefig(ruta_figura, dpi=300)
    plt.show()
    print(f"  Figura 5.2 para {params['nombre']} generada y guardada en␣
↪'{ruta_figura}'.")
```

```
Configuración completada.
Parámetros y funciones definidos.
Datos cargados y pre-procesados.

--- Iniciando generación de Figuras 5.2 ---
```




```
  Figura 5.2 para Ola 1 (Original) generada y guardada en 'Figuras
Tesis/Figura_5_2_OLA_1_Ola_1_Original_FINAL.png'.
```

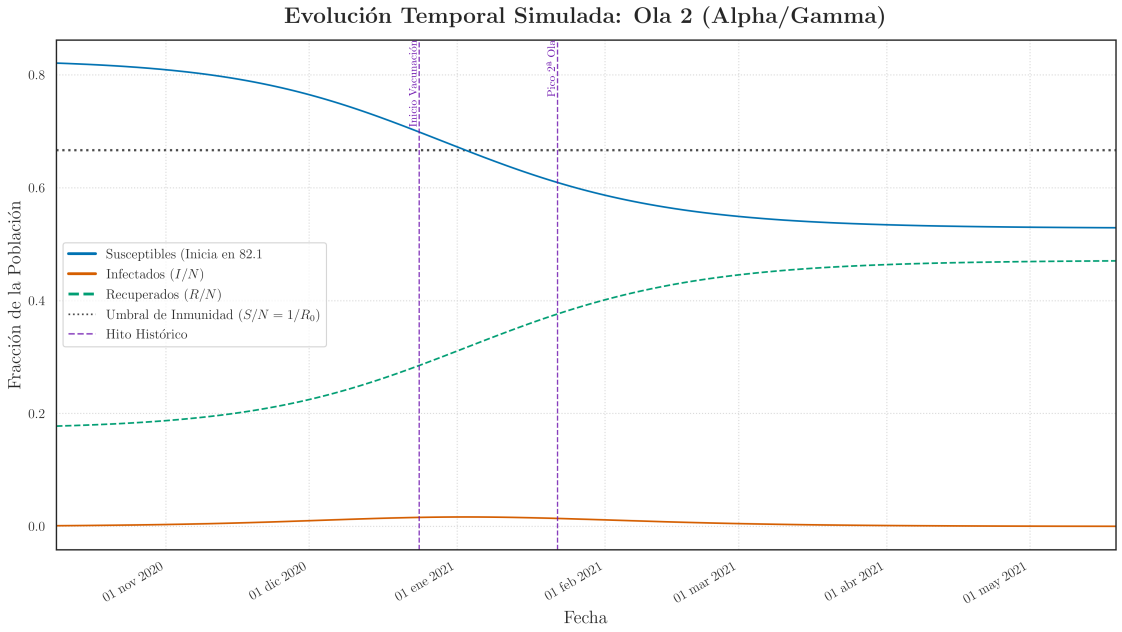


Figura 5.2 para Ola 2 (Alpha/Gamma) generada y guardada en 'Figuras Tesis/Figura_5_2_OLA_2_Ola_2_Alpha-Gamma_FINAL.png'.

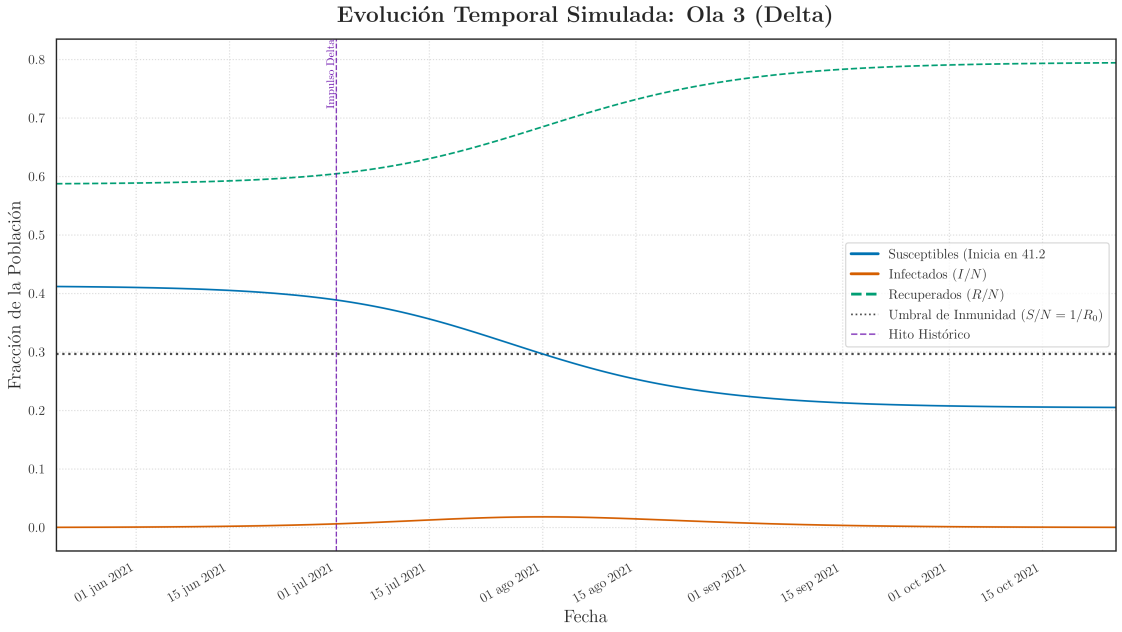


Figura 5.2 para Ola 3 (Delta) generada y guardada en 'Figuras Tesis/Figura_5_2_OLA_3_Ola_3_Delta_FINAL.png'.

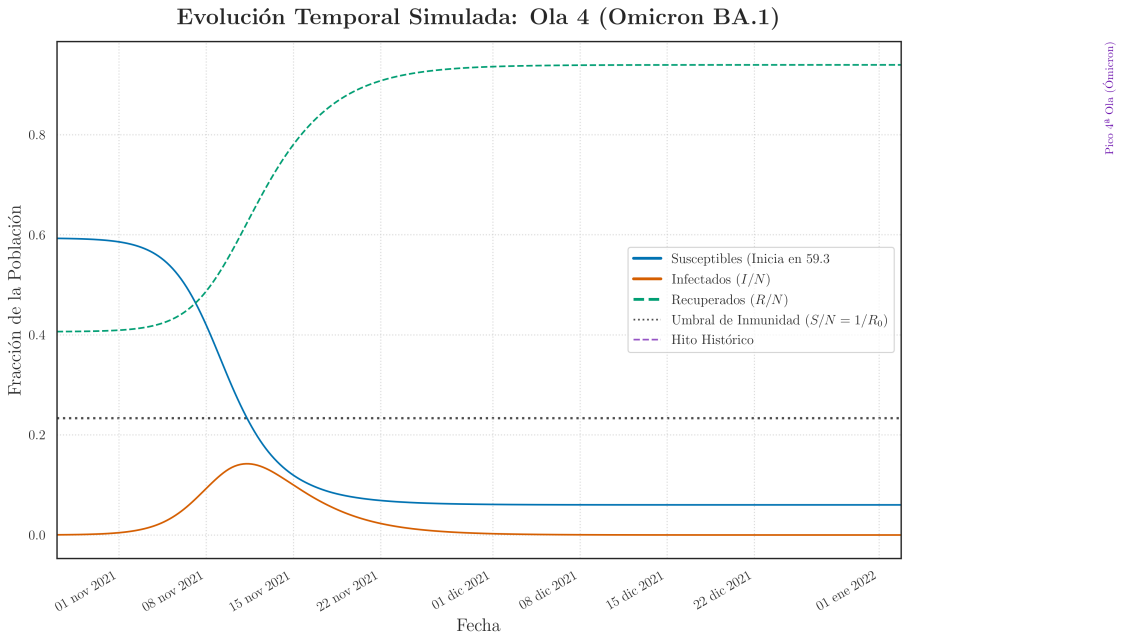


Figura 5.2 para Ola 4 (Omicron BA.1) generada y guardada en 'Figuras Tesis/Figura_5_2_OLA_4_Ola_4_Omicron_BA.1_FINAL.png'.

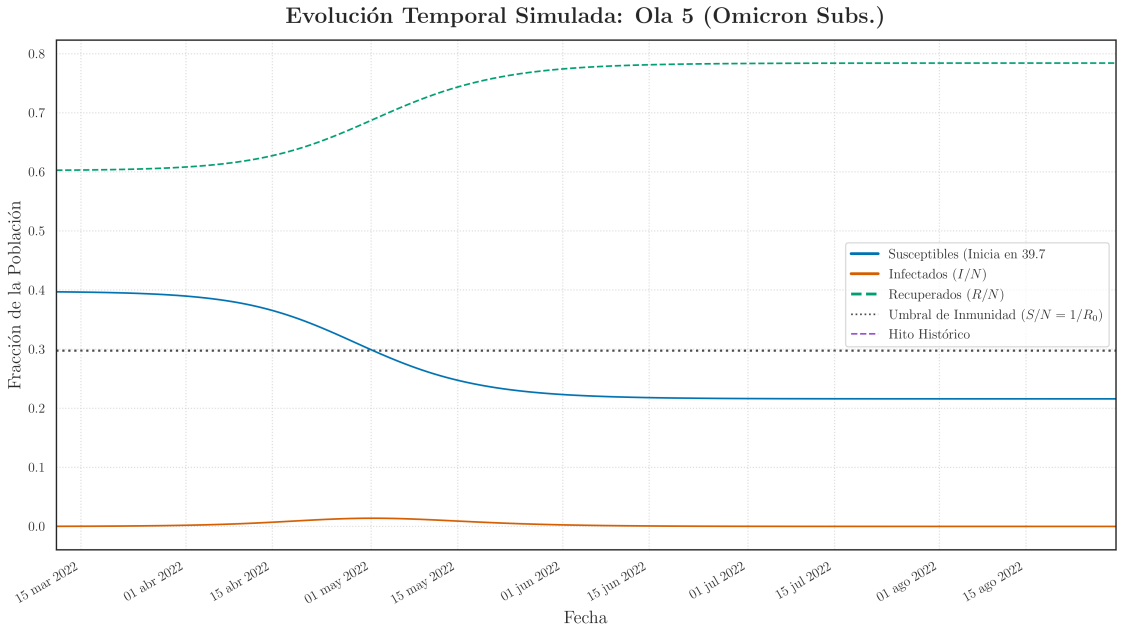


Figura 5.2 para Ola 5 (Omicron Subs.) generada y guardada en 'Figuras Tesis/Figura_5_2_OLA_5_Ola_5_Omicron_Subs._FINAL.png'.

[26]:
```
# ==============================================================================
# CELDA DEFINITIVA: FIGURA 5.3 - EVOLUCIÓN DEL NÚMERO REPRODUCTIVO EFECTIVO (Rt)
# Genera 5 figuras separadas que muestran la evolución de Rt para cada ola,
# destacando el umbral crítico y los periodos de crecimiento/decaimiento.
# ==============================================================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from scipy.integrate import solve_ivp
from matplotlib.lines import Line2D
from matplotlib.patches import Patch

# --- HITOS HISTÓRICOS Y PARÁMETROS (Asumimos que ya existen) ---
# EVENTOS_CLAVE, PARAMETROS_MAESTROS, sir_model_ivp, N_total, olas_periodos,␣
 ↪trayectorias_empiricas

# --- Función de evento para detener la simulación ---
def epidemia_terminada(t, y, R0, gamma, N):
    # Condición de parada para la simulación
    return y[1] - 100.0
epidemia_terminada.terminal = True
epidemia_terminada.direction = -1

# --- Paleta de colores ---
PALETA_RT = {
    'Rt': '#2c3e50', 'crecimiento': '#e74c3c', 'decaimiento': '#2ecc71',
    'evento': '#6a0dad', 'umbral': 'black'
}

# --- BUCLE PRINCIPAL PARA GENERAR 5 FIGURAS SEPARADAS ---
print("\nGenerando 5 figuras para la Evolución de Rt...")
for i in range(len(PARAMETROS_MAESTROS)):

    fig, ax = plt.subplots(figsize=(14, 8))
    ola_idx = i + 1
    params = PARAMETROS_MAESTROS[ola_idx]
    fecha_inicio_ola = olas_periodos[i]['inicio']
    eventos_ola = [e for e in EVENTOS_CLAVE if e['ola'] == ola_idx]

    # --- 1. Simulación (necesaria para obtener S(t)) ---
    gamma = 1.0 / params['D']
    s0_abs = params['S0_N'] * N_total
    i0_abs = trayectorias_empiricas[i]['I_real_abs'].iloc[0]
    r0_abs = N_total - s0_abs - i0_abs

    sol = solve_ivp(
```

```
        sir_model_ivp, (0, 1000), [s0_abs, i0_abs, r0_abs],
        dense_output=True, events=epidemia_terminada,
        args=(params['R0'], gamma, N_total), method='RK45'
    )

    t_eval_sim = np.linspace(sol.t[0], sol.t[-1], 500)
    y_eval_sim = sol.sol(t_eval_sim)
    fechas_simulacion = fecha_inicio_ola + pd.to_timedelta(t_eval_sim, unit='D')

    # --- 2. Cálculo de Rt ---
    S_frac_t = y_eval_sim[0] / N_total
    Rt = params['R0'] * S_frac_t

    # --- 3. Graficar Rt y sus componentes ---
    # Curva principal de Rt
    ax.plot(fechas_simulacion, Rt, color=PALETA_RT['Rt'], lw=3)

    # Umbral Crítico
    ax.axhline(y=1, color=PALETA_RT['umbral'], linestyle='--', linewidth=2,␣
↪label='Umbral Crítico ($R_t=1$)')

    # Sombreado de áreas de crecimiento y decaimiento
    ax.fill_between(fechas_simulacion, 1, Rt, where=Rt >= 1,␣
↪color=PALETA_RT['crecimiento'], alpha=0.2, label='Periodo de Crecimiento␣
↪($R_t>1$)')
    ax.fill_between(fechas_simulacion, 1, Rt, where=Rt <= 1,␣
↪color=PALETA_RT['decaimiento'], alpha=0.2, label='Periodo de Decaimiento␣
↪($R_t<1$)')

    # Hitos Históricos
    for evento in eventos_ola:
        fecha_evento = pd.to_datetime(evento['fecha'])
        if fechas_simulacion.min() <= fecha_evento <= fechas_simulacion.max():
            ax.axvline(x=fecha_evento, color=PALETA_RT['evento'], linestyle=':
↪', lw=1.5, alpha=0.9)

    # --- 4. Formato y Títulos ---
    ax.set_title(fr"\textbf{{Evolución del Número Reproductivo Efectivo ($R_t$):
↪ {params['nombre']}}}", fontsize=20, pad=15)
    ax.grid(True, linestyle=':', alpha=0.7)
    ax.set_xlim(fechas_simulacion.min(), fechas_simulacion.max())
    ax.set_ylabel('Número Reproductivo Efectivo ($R_t$)', fontsize=16)
    ax.set_xlabel('Fecha', fontsize=16)

    ax.xaxis.set_major_locator(mdates.AutoDateLocator(minticks=5, maxticks=10))
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
    plt.setp(ax.get_xticklabels(), rotation=30, ha="right")
```

```
    # Caja de información
    Rt_inicial = params['R0'] * params['S0_N']
    info_text = fr"$R_t(0) = R_0 \times S_0/N = {params['R0']:.2f} \times␣
↪{params['S0_N']:.3f} \approx {Rt_inicial:.2f}$"
    ax.text(0.02, 0.02, info_text, transform=ax.transAxes, fontsize=14,␣
↪va='bottom', ha='left',
            bbox=dict(boxstyle='round,pad=0.5', facecolor='white', alpha=0.8,␣
↪ec='grey'))

    # Leyenda
    handles, labels = ax.get_legend_handles_labels()
    # Añadir proxy para los hitos
    if eventos_ola:
        handles.append(Line2D([0], [0], color=PALETA_RT['evento'], linestyle=':
↪', label='Hito Histórico'))
    ax.legend(handles=handles, loc='upper right', fontsize=12)

    # --- 5. Guardado ---
    fig.tight_layout()
    nombre_ola_archivo = params['nombre'].replace(' ', '_').replace('(', '').
↪replace(')', '').replace('/', '-')
    ruta_figura = f'Figuras Tesis/Figura_5_3_OLA_{i+1}_{nombre_ola_archivo}_Rt.
↪png'
    fig.savefig(ruta_figura, dpi=300)
    plt.show()
    print(f"  Figura de Rt para {params['nombre']} generada y guardada en␣
↪'{ruta_figura}'.")
```

```
Generando 5 figuras para la Evolución de Rt…
```

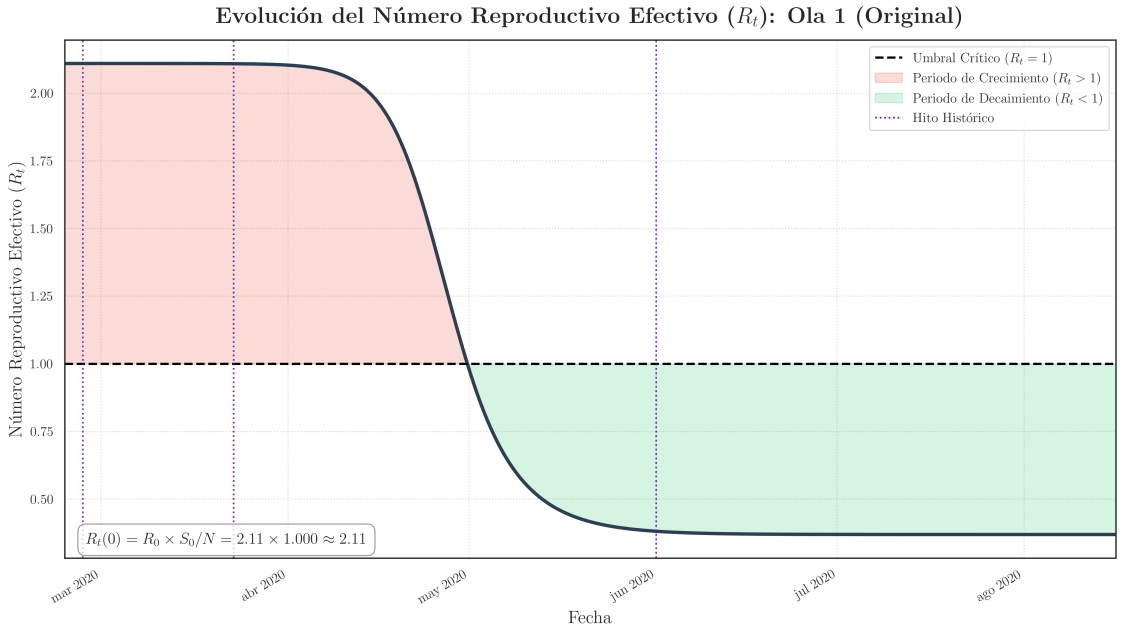


Figura de Rt para Ola 1 (Original) generada y guardada en 'Figuras Tesis/Figura_5_3_OLA_1_Ola_1_Original_Rt.png'.

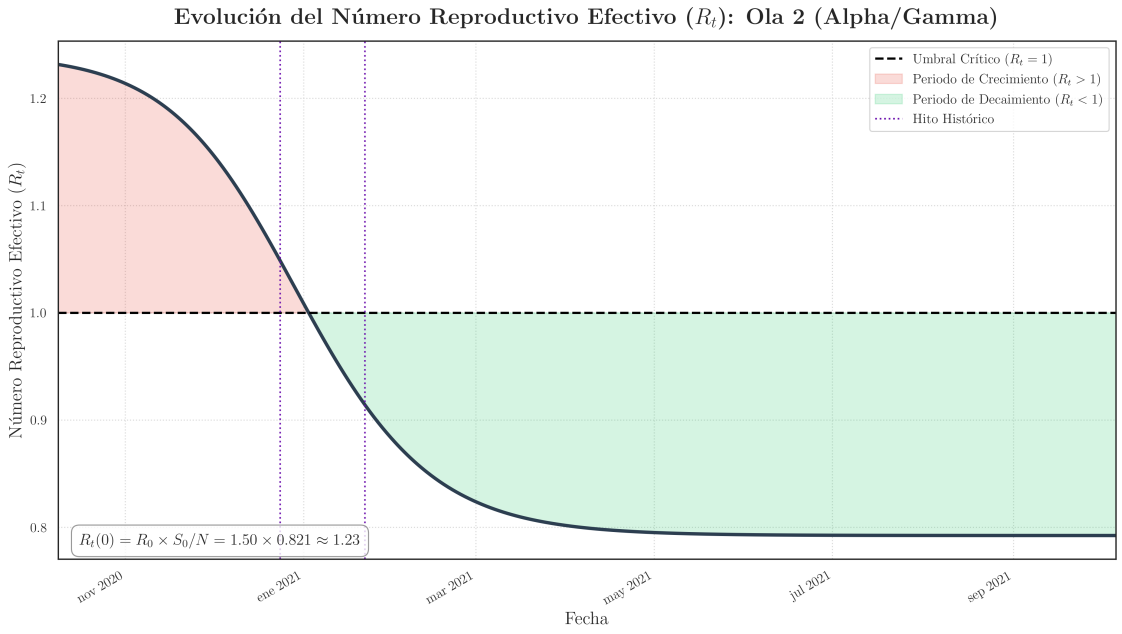


Figura de Rt para Ola 2 (Alpha/Gamma) generada y guardada en 'Figuras Tesis/Figura_5_3_OLA_2_Ola_2_Alpha-Gamma_Rt.png'.

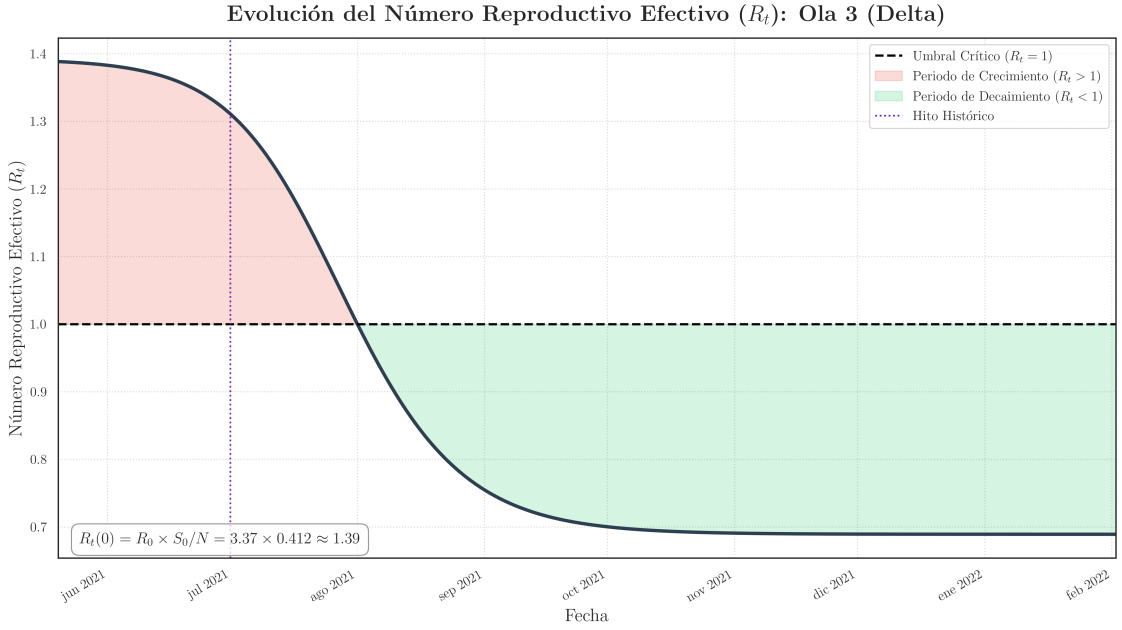


Figura de Rt para Ola 3 (Delta) generada y guardada en 'Figuras Tesis/Figura_5_3_OLA_3_Ola_3_Delta_Rt.png'.

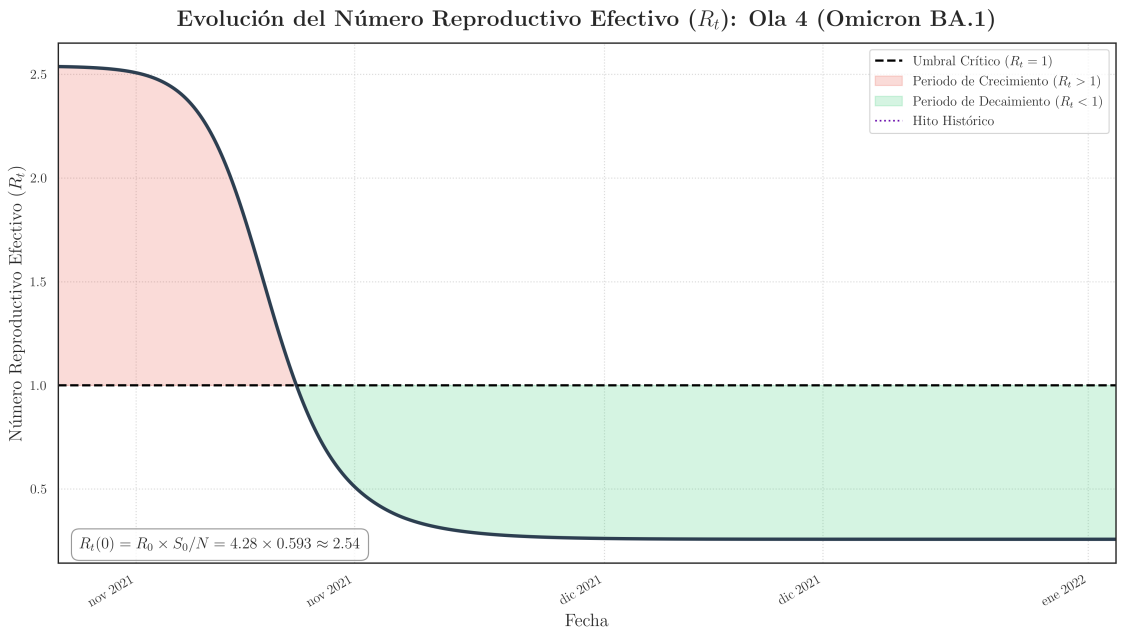


Figura de Rt para Ola 4 (Omicron BA.1) generada y guardada en 'Figuras Tesis/Figura_5_3_OLA_4_Ola_4_Omicron_BA.1_Rt.png'.

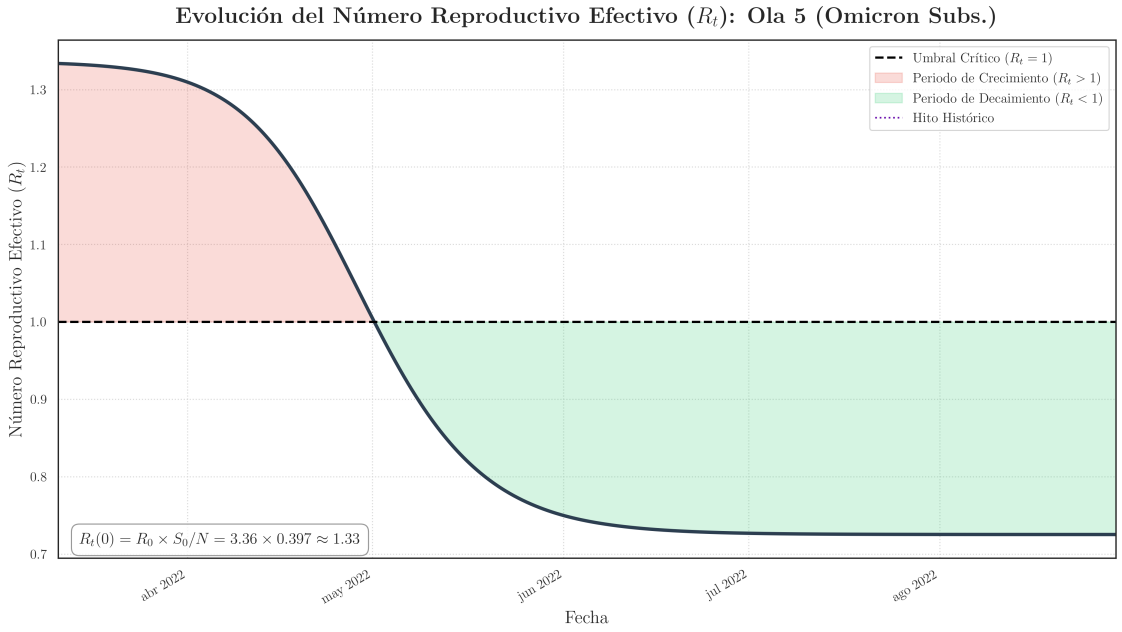


Figura de Rt para Ola 5 (Omicron Subs.) generada y guardada en 'Figuras Tesis/Figura_5_3_OLA_5_Ola_5_Omicron_Subs._Rt.png'.

[27]:
```
# ==============================================================================
# FIGURA 5.1: ESQUEMA CONCEPTUAL DE PERTURBACIÓN-RELAJACIÓN (DISEÑO FINAL)
# ==============================================================================
# Este script genera una visualización de calidad editorial para ilustrar el
# concepto de un sistema que es perturbado y se relaja hacia un nuevo
# estado de equilibrio, con anotaciones ajustadas al texto de la tesis.
# ==============================================================================

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.collections import LineCollection
import matplotlib.patheffects as path_effects
import os

# ------------------------------------------------------------------------------
# 1. CONFIGURACIÓN GLOBAL Y PALETA DE COLORES DE PUBLICACIÓN
# ------------------------------------------------------------------------------
# Paleta de colores refinada para una estética sobria y académica.
PALETA_EDITORIAL = {
    'fondo': '#FFFFFF',
    'texto_principal': '#222222',
    'texto_secundario': '#555555',
    'paisaje': '#0072B2',
```

```
    'paisaje_sombra': '#dbeffc',
    'trayectoria': '#D55E00',
    'perturbacion': '#E69F00',
    'estado': '#FFFFFF',
    'estado_borde': '#222222',
    'ejes': '#888888'
}

# Configuración de Matplotlib para una estética de publicación (usando LaTeX).
plt.rcParams.update({
    'font.family': 'serif',
    'font.serif': ['Computer Modern Roman'],
    'text.usetex': True,
    'text.latex.preamble':␣
 ↪r'\usepackage{amsmath}\usepackage{amssymb}\usepackage{sansmath}\sansmath',
    'figure.dpi': 300,
    'savefig.dpi': 300,
    'savefig.bbox': 'tight',
})

# ------------------------------------------------------------------------------
# 2. FUNCIONES AUXILIARES PARA EL DIBUJO
# ------------------------------------------------------------------------------

def definir_paisaje_potencial(x):
    """
    Define la forma del paisaje de potencial usando una combinación de␣
 ↪gaussianas.
    Crea dos valles que representan los estados de equilibrio.
    """
    g1 = -1.5 * np.exp(-((x - 2.5)**2) / 1.8)
    g2 = -2.0 * np.exp(-((x - 9.5)**2) / 2.5)
    base = 0.05 * x + 4
    return g1 + g2 + base

def dibujar_paisaje_y_ejes(ax, x_range):
    """Dibuja el paisaje de potencial con un sombreado suave y ejes␣
 ↪contextuales."""
    y_paisaje = definir_paisaje_potencial(x_range)
    ax.plot(x_range, y_paisaje, color=PALETA_EDITORIAL['paisaje'], linewidth=2.
 ↪0, zorder=2)
    ax.fill_between(x_range, y_paisaje, 0,␣
 ↪color=PALETA_EDITORIAL['paisaje_sombra'], zorder=1, alpha=0.6)
    ax.axhline(y=1, xmin=0.05, xmax=0.95, color=PALETA_EDITORIAL['ejes'], lw=0.
 ↪75, ls='--', zorder=0)
```

```
    ax.axvline(x=0.5, ymin=0.1, ymax=0.9, color=PALETA_EDITORIAL['ejes'], lw=0.
↪75, ls='--', zorder=0)
    ax.text(0.4, 7.5, r'Energía Potencial del Sistema', rotation=90,␣
↪va='center', ha='right',
            color=PALETA_EDITORIAL['texto_secundario'], fontsize=12)
    ax.text(6.0, 0.7, r'Espacio de Estados (ej. Fracción de Susceptibles)',␣
↪va='center', ha='center',
            color=PALETA_EDITORIAL['texto_secundario'], fontsize=12)

def dibujar_estados_equilibrio(ax):
    """Dibuja las partículas en los valles, representando los estados E1 y E2.
↪"""
    estilo_particula = dict(radius=0.22, zorder=5,␣
↪edgecolor=PALETA_EDITORIAL['estado_borde'],
                            linewidth=1.2,␣
↪facecolor=PALETA_EDITORIAL['estado'])
    halo_particula = [path_effects.withStroke(linewidth=2,␣
↪foreground=PALETA_EDITORIAL['estado'])]
    x1, y1 = 2.5, definir_paisaje_potencial(2.5)
    circ1 = mpatches.Circle((x1, y1), **estilo_particula)
    circ1.set_path_effects(halo_particula)
    ax.add_patch(circ1)
    x2, y2 = 9.5, definir_paisaje_potencial(9.5)
    circ2 = mpatches.Circle((x2, y2), **estilo_particula)
    circ2.set_path_effects(halo_particula)
    ax.add_patch(circ2)
    return (x1, y1), (x2, y2)

def dibujar_trayectoria_y_perturbacion(ax, pos_e1, pos_e2):
    """Dibuja la perturbación y la trayectoria de relajación."""
    ax.annotate("", xy=(pos_e1[0] + 1.0, pos_e1[1] + 0.8), xytext=(pos_e1[0] +␣
↪2.8, pos_e1[1] + 2.0),
                arrowprops=dict(arrowstyle="fancy,head_length=0.6,head_width=0.
↪4,tail_width=0.3",
                                color=PALETA_EDITORIAL['perturbacion'],
                                connectionstyle="arc3,rad=-0.2", lw=0,␣
↪shrinkA=0, shrinkB=0), zorder=3)
    x_traj = np.linspace(pos_e1[0], pos_e2[0], 200)
    y_traj = definir_paisaje_potencial(x_traj) + 0.25 * np.sin(np.pi * (x_traj␣
↪- pos_e1[0]) / (pos_e2[0] - pos_e1[0]))**0.5
    points = np.array([x_traj, y_traj]).T.reshape(-1, 1, 2)
    segments = np.concatenate([points[:-1], points[1:]], axis=1)
    norm = plt.Normalize(0, 1)
    lc = LineCollection(segments, cmap='autumn', norm=norm)
    lc.set_array(np.linspace(0.3, 1.0, len(x_traj)))
    lc.set_linewidth(2.5)
```

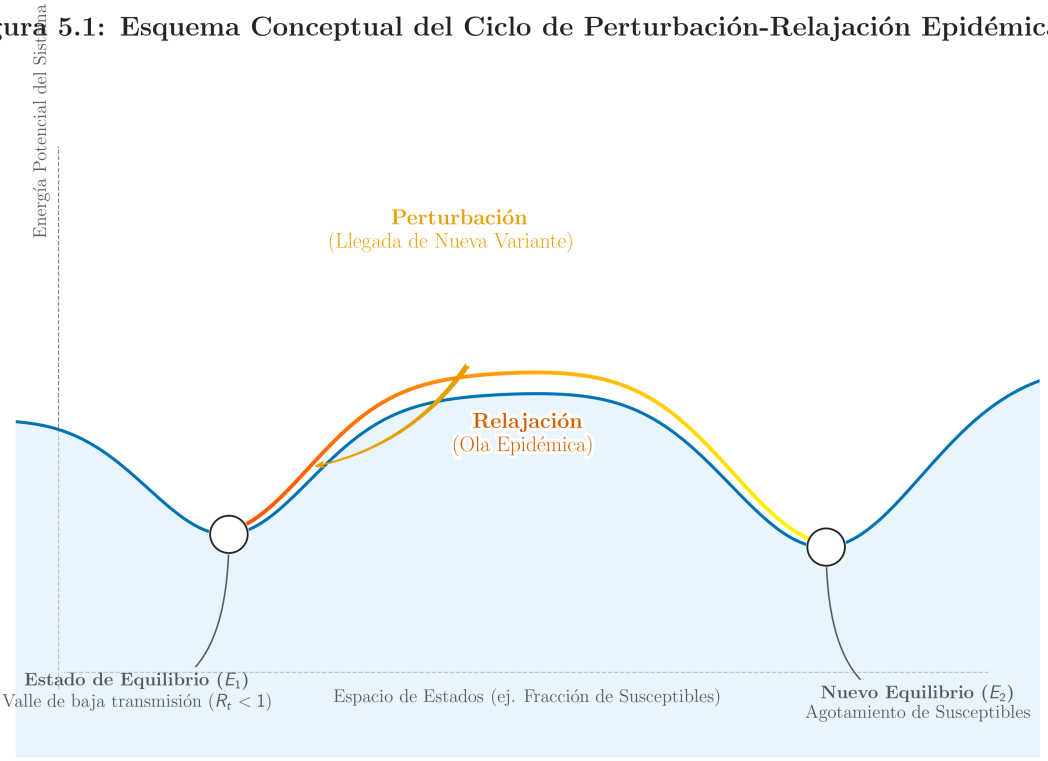
```
    lc.set_linestyle((0, (4, 2)))
    lc.set_zorder(4)
    ax.add_collection(lc)

def agregar_anotaciones(ax, pos_e1, pos_e2):
    """Añade etiquetas y textos explicativos alineados con el texto de la tesis.
    """
    halo = [path_effects.withStroke(linewidth=4,␣
    ↪foreground=PALETA_EDITORIAL['fondo'])]

    # Anotación del Estado 1 (Equilibrio Inicial)
    ax.annotate(r'\textbf{Estado de Equilibrio ($E_1$)}' + '\n' + r'Valle de␣
    ↪baja transmisión ($R_t < 1$)',
                xy=pos_e1, xytext=(-60, -90), textcoords='offset points',
                ha='center', va='top', fontsize=12,␣
    ↪color=PALETA_EDITORIAL['texto_secundario'],
                arrowprops=dict(arrowstyle="-",␣
    ↪color=PALETA_EDITORIAL['texto_secundario'],
                                connectionstyle="angle3,angleA=0,angleB=90",␣
    ↪lw=1.0))

    # Anotación del Estado 2 (Nuevo Equilibrio)
    ax.annotate(r'\textbf{Nuevo Equilibrio ($E_2$)}' + '\n' + r'Agotamiento de␣
    ↪Susceptibles',
                xy=pos_e2, xytext=(60, -90), textcoords='offset points',
                ha='center', va='top', fontsize=12,␣
    ↪color=PALETA_EDITORIAL['texto_secundario'],
                arrowprops=dict(arrowstyle="-",␣
    ↪color=PALETA_EDITORIAL['texto_secundario'],
                                connectionstyle="angle3,angleA=0,angleB=90",␣
    ↪lw=1.0))

    # Anotación de la Perturbación
    ax.text(5.2, 6.2, r'\textbf{Perturbación}' + '\n' + r'(Llegada de Nueva␣
    ↪Variante)', fontsize=14,
            color=PALETA_EDITORIAL['perturbacion'], ha='center', va='center',␣
    ↪path_effects=halo)

    # Anotación de la Trayectoria
    ax.text(6.0, 3.8, r'\textbf{Relajación}' + '\n' + r'(Ola Epidémica)',␣
    ↪fontsize=14,
            color=PALETA_EDITORIAL['trayectoria'], ha='center', va='center',␣
    ↪path_effects=halo)

# ------------------------------------------------------------------------
# 3. FUNCIÓN PRINCIPAL Y GENERACIÓN DE LA FIGURA
```

```python
# ------------------------------------------------------------------------------
def generar_figura_conceptual():
    """Orquesta la creación y guardado de la figura."""
    print("Generando Figura 5.1 (Diseño Editorial Final)...")

    fig, ax = plt.subplots(figsize=(12, 8))
    fig.set_facecolor(PALETA_EDITORIAL['fondo'])
    ax.set_facecolor(PALETA_EDITORIAL['fondo'])

    ax.set_xlim(0, 12)
    ax.set_ylim(0, 8)
    ax.axis('off')

    x_range = np.linspace(0, 12, 500)
    dibujar_paisaje_y_ejes(ax, x_range)
    pos_e1, pos_e2 = dibujar_estados_equilibrio(ax)
    dibujar_trayectoria_y_perturbacion(ax, pos_e1, pos_e2)
    agregar_anotaciones(ax, pos_e1, pos_e2)

    fig.suptitle(r'\textbf{Figura 5.1: Esquema Conceptual del Ciclo de␣
 ↪Perturbación-Relajación Epidémica}',
                 fontsize=18, color=PALETA_EDITORIAL['texto_principal'], y=0.95)

    directorio_salida = 'Figuras Tesis'
    if not os.path.exists(directorio_salida):
        os.makedirs(directorio_salida)

    ruta_pdf = os.path.join(directorio_salida,␣
 ↪'Figura_5_1_Esquema_Editorial_Final.pdf')
    ruta_png = os.path.join(directorio_salida,␣
 ↪'Figura_5_1_Esquema_Editorial_Final.png')

    plt.savefig(ruta_pdf, facecolor=fig.get_facecolor(), edgecolor='none',␣
 ↪pad_inches=0.1)
    plt.savefig(ruta_png, facecolor=fig.get_facecolor(), edgecolor='none',␣
 ↪pad_inches=0.1)

    print(f"  Figura de alta calidad guardada en '{ruta_pdf}' y '{ruta_png}'")

    plt.show()

# --- Ejecutar el script ---
if __name__ == '__main__':
    generar_figura_conceptual()
```

```
Generando Figura 5.1 (Diseño Editorial Final)…
  Figura de alta calidad guardada en 'Figuras
```

```
Tesis/Figura_5_1_Esquema_Editorial_Final.pdf' y 'Figuras
Tesis/Figura_5_1_Esquema_Editorial_Final.png'
```

Figura 5.1: Esquema Conceptual del Ciclo de Perturbación-Relajación Epidémica



[28]:
```python
# ==============================================================================
# CELDA DEFINITIVA: CAPÍTULO 6 - SIMULACIÓN DEL SISTEMA DE ALERTA TEMPRANA
# Versión final y sincronizada con Parámetros Maestros, condiciones iniciales
# realistas y duraciones de ola correctas.
# ==============================================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
import matplotlib.dates as mdates
import re

# --- PARÁMETROS, FUNCIONES Y DATOS (Asumimos que ya existen en memoria) ---
# PARAMETROS_MAESTROS, EVENTOS_CLAVE, N_total, sir_model_ivp,
# olas_periodos, trayectorias_empiricas

# --- Paleta de colores y configuración del sistema de alerta ---
PALETA_ALERTA = {
    'curva_I': 'black', 'velocidad': '#ffa600', 'aceleracion': '#bc5090',
    'evento': '#404040'
```

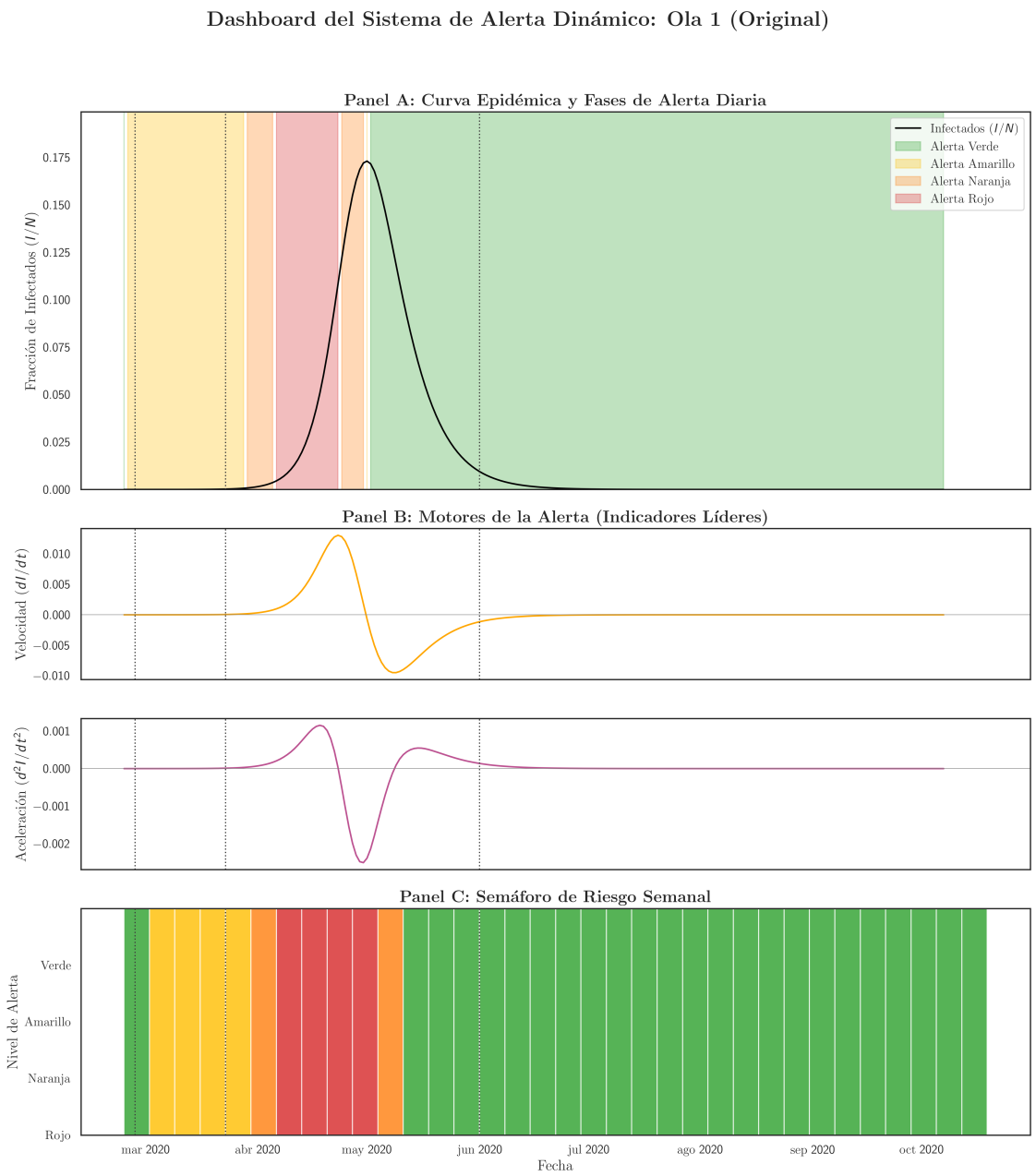
```
}
ALERT_LEVELS = {
    1: ("Verde", "#2ca02c"), 2: ("Amarillo", "#ffbf00"),
    3: ("Naranja", "#ff7f0e"), 4: ("Rojo", "#d62728")
}

# --- Funciones específicas del sistema de alerta ---
def calculate_alert_levels(t, S_t_frac, I_t_frac, R0, gamma):
    Rt = R0 * S_t_frac
    beta = R0 * gamma
    # Las derivadas deben calcularse con las fracciones para mantener la␣
 ↪consistencia
    speed_I = beta * S_t_frac * I_t_frac - gamma * I_t_frac
    accel_I = np.gradient(speed_I, t)

    idx_growth = np.where(speed_I > 0)[0]
    # Umbrales estadísticos basados en la fase de crecimiento
    tau_speed = np.percentile(speed_I[idx_growth], 50) if len(idx_growth) > 1␣
 ↪else np.inf
    tau_accel = np.percentile(accel_I[idx_growth], 75) if len(idx_growth) > 1␣
 ↪else np.inf

    alert_levels = np.ones(len(t), dtype=int)
    for i in range(1, len(t)):
        alert_levels[i] = alert_levels[i-1]
        # Lógica de escalada
        if alert_levels[i-1] == 1 and Rt[i] > 1: alert_levels[i] = 2
        elif alert_levels[i-1] == 2 and speed_I[i] > tau_speed: alert_levels[i]␣
 ↪= 3
        elif alert_levels[i-1] == 3 and accel_I[i] > tau_accel: alert_levels[i]␣
 ↪= 4
        # Lógica de desescalada (simplificada)
        if alert_levels[i-1] == 4 and accel_I[i] < 0: alert_levels[i] = 3
        elif alert_levels[i-1] == 3 and speed_I[i] < 0: alert_levels[i] = 2
        elif alert_levels[i-1] == 2 and Rt[i] < 1: alert_levels[i] = 1

    return alert_levels, speed_I, accel_I

def assign_weekly_alert(dates, daily_alerts):
    df = pd.DataFrame({'date': dates, 'alert': daily_alerts})
    # Usar 'W-MON' para agrupar por semana de lunes a domingo
    weekly_summary = df.groupby(pd.Grouper(key='date', freq='W-MON'))['alert'].
 ↪max()
    return weekly_summary

# --- BUCLE PRINCIPAL PARA GENERAR 5 DASHBOARDS DE ALERTA ---
```

```
print("\nGenerando 5 dashboards para el sistema de alerta...")
for i in range(len(PARAMETROS_MAESTROS)):

    ola_idx = i + 1
    params = PARAMETROS_MAESTROS[ola_idx]
    fecha_inicio_ola = olas_periodos[i]['inicio']
    fecha_fin_ola = olas_periodos[i]['fin']

    # Simulación con inicio realista y duración de ola correcta
    gamma = 1.0 / params['D']
    s0_abs = params['S0_N'] * N_total
    i0_abs = trayectorias_empiricas[i]['I_real_abs'].iloc[0]
    r0_abs = N_total - s0_abs - i0_abs

    duracion_dias = (fecha_fin_ola - fecha_inicio_ola).days
    t_span = (0, duracion_dias)
    t_eval = np.arange(t_span[0], t_span[1])

    sol = solve_ivp(sir_model_ivp, t_span, [s0_abs, i0_abs, r0_abs],
                    t_eval=t_eval, args=(params['R0'], gamma, N_total),␣
 ↪method='RK45')

    t, S_t, I_t = sol.t, sol.y[0], sol.y[1]
    eje_fechas = fecha_inicio_ola + pd.to_timedelta(t, unit='D')

    # Calcular niveles de alerta
    alert_levels, speed_I, accel_I = calculate_alert_levels(t, S_t/N_total, I_t/
 ↪N_total, params['R0'], gamma)
    weekly_alerts = assign_weekly_alert(eje_fechas, alert_levels)

    # --- Creación de la Gráfica ---
    fig, (ax1, ax2, ax3, ax4) = plt.subplots(4, 1, figsize=(14, 16),␣
 ↪sharex=True, gridspec_kw={'height_ratios': [2.5, 1, 1, 1.5]})
    fig.suptitle(fr"\textbf{{Dashboard del Sistema de Alerta Dinámico:␣
 ↪{params['nombre']}}}", y=0.98, fontsize=20)

    # Panel A: Curva Epidémica y Fases de Alerta
    ax1.set_title(r'\textbf{Panel A: Curva Epidémica y Fases de Alerta Diaria}')
    ax1.plot(eje_fechas, I_t / N_total, color=PALETA_ALERTA['curva_I'],␣
 ↪label='Infectados ($I/N$)')
    y_max = np.max(I_t / N_total) * 1.15
    ax1.set_ylim(0, y_max)
    for level, (name, color) in ALERT_LEVELS.items():
        ax1.fill_between(eje_fechas, 0, y_max, where=alert_levels == level,␣
 ↪color=color, alpha=0.3, label=f'Alerta {name}')
```

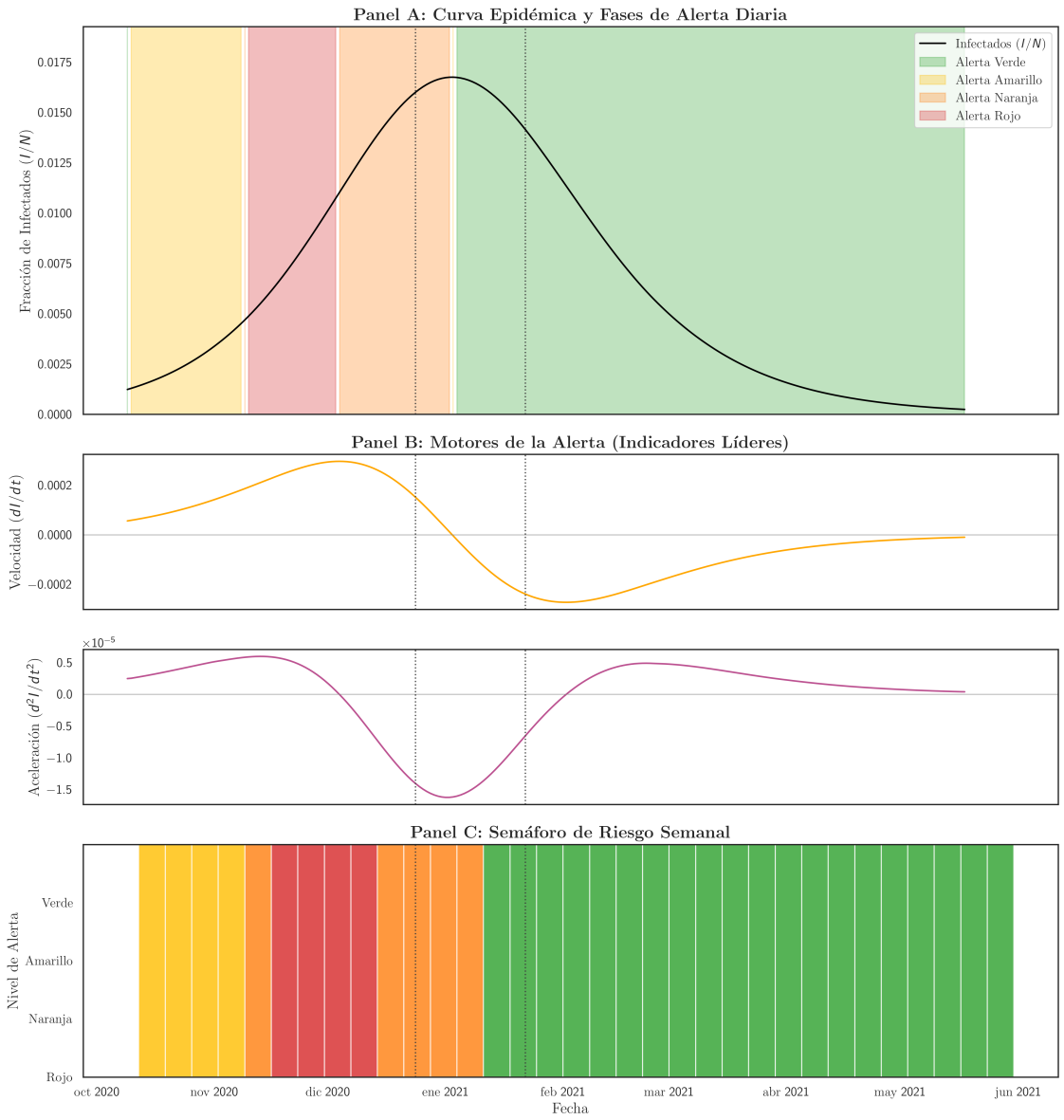
```
    ax1.set_ylabel('Fracción de Infectados ($I/N$)'); ax1.legend(loc='upper␣
↪right')

    # Panel B: Motores de la Alerta
    ax2.set_title(r'\textbf{Panel B: Motores de la Alerta (Indicadores␣
↪Líderes)}')
    ax2.plot(eje_fechas, speed_I, color=PALETA_ALERTA['velocidad']); ax2.
↪axhline(0, color='grey', lw=0.5)
    ax2.set_ylabel('Velocidad ($dI/dt$)')
    ax3.plot(eje_fechas, accel_I, color=PALETA_ALERTA['aceleracion']); ax3.
↪axhline(0, color='grey', lw=0.5)
    ax3.set_ylabel('Aceleración ($d^2I/dt^2$)')

    # Panel C: Semáforo Semanal
    ax4.set_title(r'\textbf{Panel C: Semáforo de Riesgo Semanal}')
    for week_start_date, alert_level in weekly_alerts.items():
        ax4.axvspan(week_start_date, week_start_date + pd.Timedelta(days=7),
                    facecolor=ALERT_LEVELS[alert_level][1], alpha=0.8,␣
↪ec='white')
    ax4.set_yticks(list(ALERT_LEVELS.keys())); ax4.set_yticklabels([name for␣
↪name, _ in ALERT_LEVELS.values()])
    ax4.set_ylabel('Nivel de Alerta'); ax4.invert_yaxis()

    # Hitos históricos relevantes
    eventos_ola = [e for e in EVENTOS_CLAVE if e['ola'] == ola_idx]
    for evento in eventos_ola:
        fecha_evento = pd.to_datetime(evento['fecha'])
        if fecha_inicio_ola <= fecha_evento <= fecha_fin_ola:
            for ax_panel in (ax1, ax2, ax3, ax4):
                ax_panel.axvline(fecha_evento, color=PALETA_ALERTA['evento'],␣
↪linestyle=':', lw=1.2)

    ax4.xaxis.set_major_locator(mdates.MonthLocator(interval=1 if duracion_dias␣
↪< 400 else 2))
    ax4.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
    ax4.set_xlabel('Fecha')

    fig.tight_layout(rect=[0, 0, 1, 0.95])

    # Guardado
    safe_name = params['nombre'].replace(' ', '_').replace('(', '').
↪replace(')', '').replace('/', '-')
    # Corregir numeración de figuras
    ruta_figura_pdf = f'Figuras Tesis/Figura_6_{ola_idx}_Alerta_{safe_name}.png'
    fig.savefig(ruta_figura_pdf)
    plt.show()
```

```
    plt.close(fig)
    print(f" -> Dashboard para '{params['nombre']}' guardado.")

print("\n  Todas las figuras del sistema de alerta han sido generadas y␣
 ↪guardadas.")
```

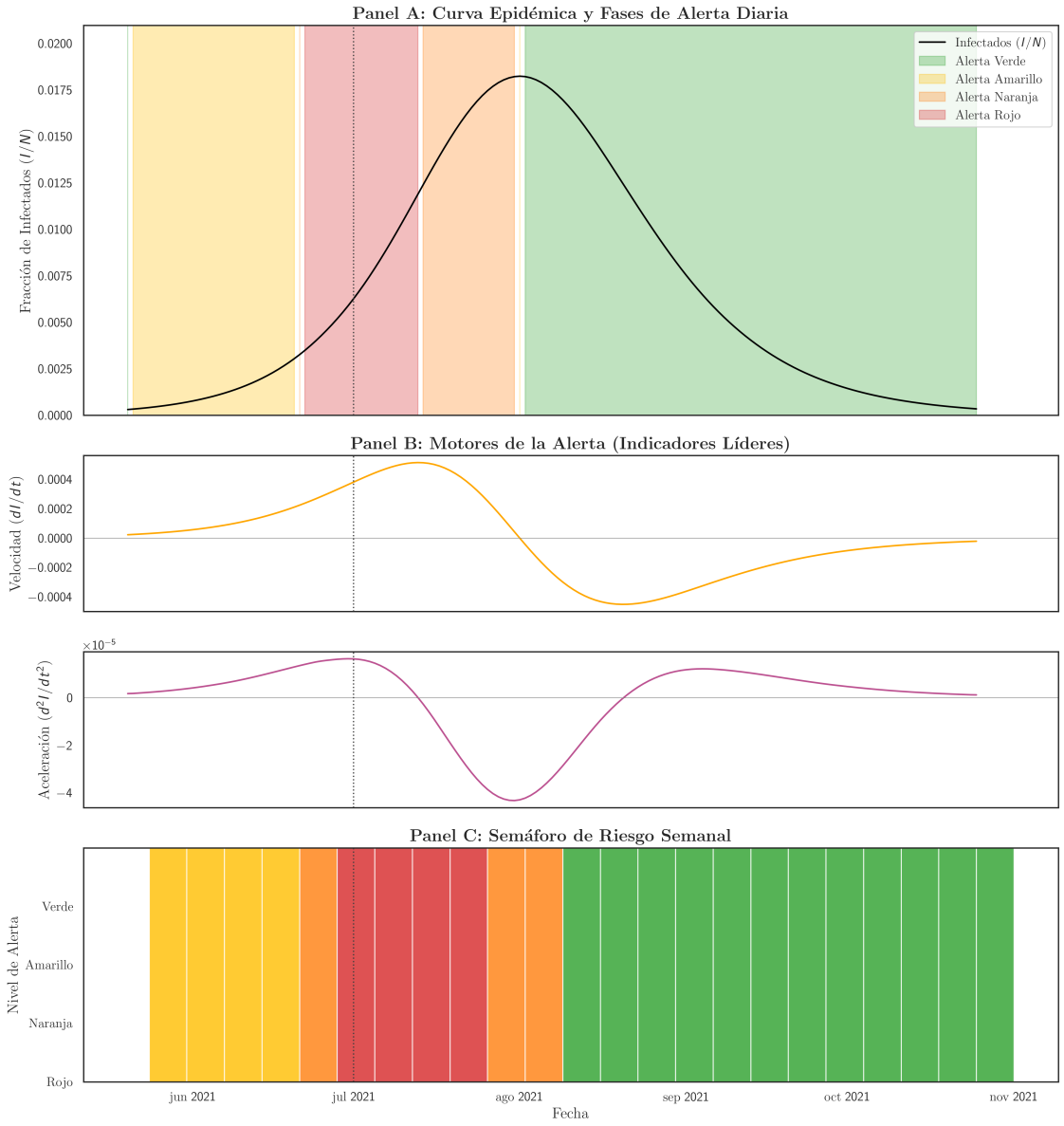
```
Generando 5 dashboards para el sistema de alerta…
```



```
-> Dashboard para 'Ola 1 (Original)' guardado.
```



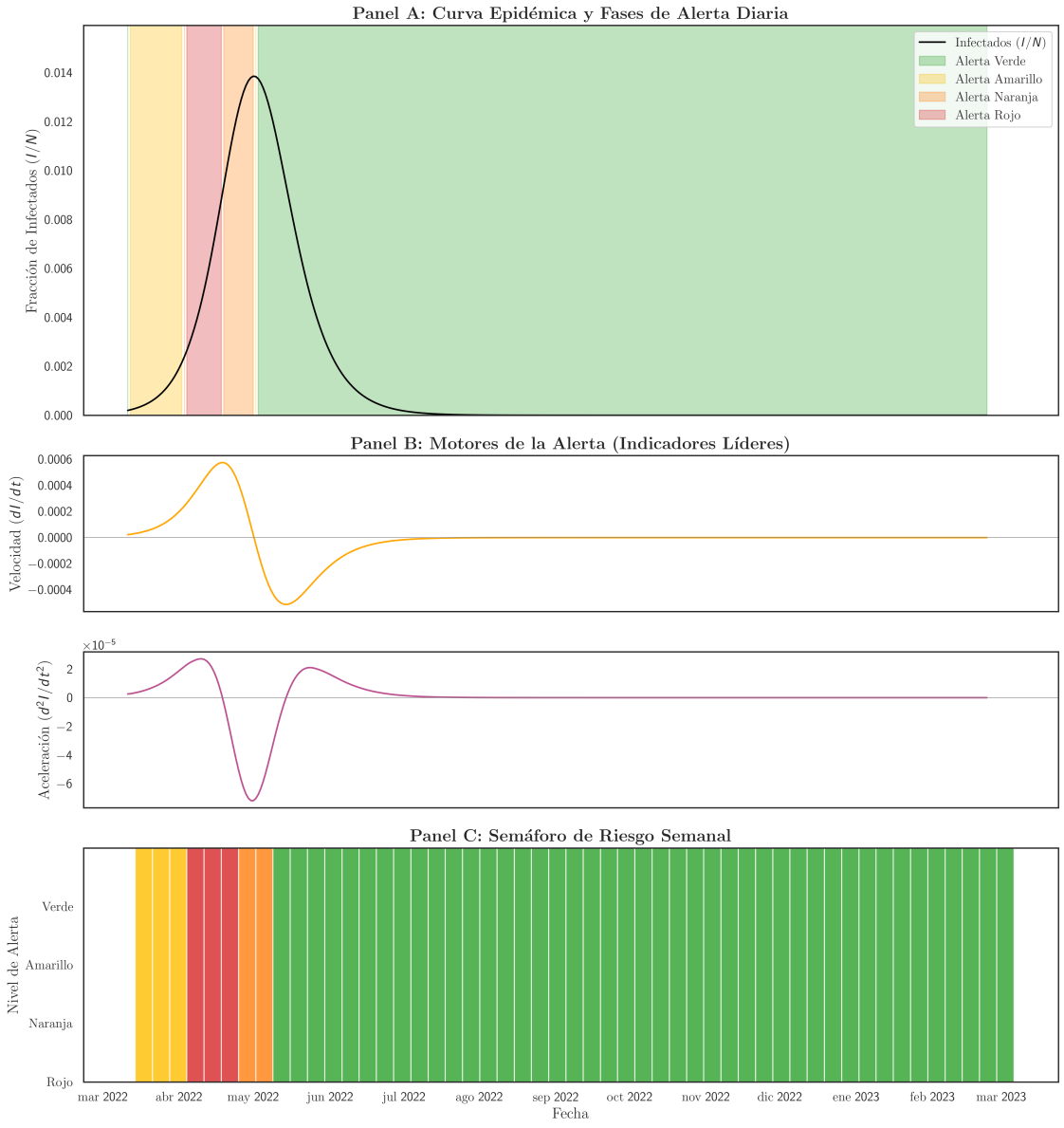

```
-> Dashboard para 'Ola 2 (Alpha/Gamma)' guardado.
```




```
-> Dashboard para 'Ola 3 (Delta)' guardado.
```

Dashboard del Sistema de Alerta Dinámico: Ola 4 (Omicron BA.1)

-> Dashboard para 'Ola 4 (Omicron BA.1)' guardado.



-> Dashboard para 'Ola 5 (Omicron Subs.)' guardado.

Todas las figuras del sistema de alerta han sido generadas y guardadas.

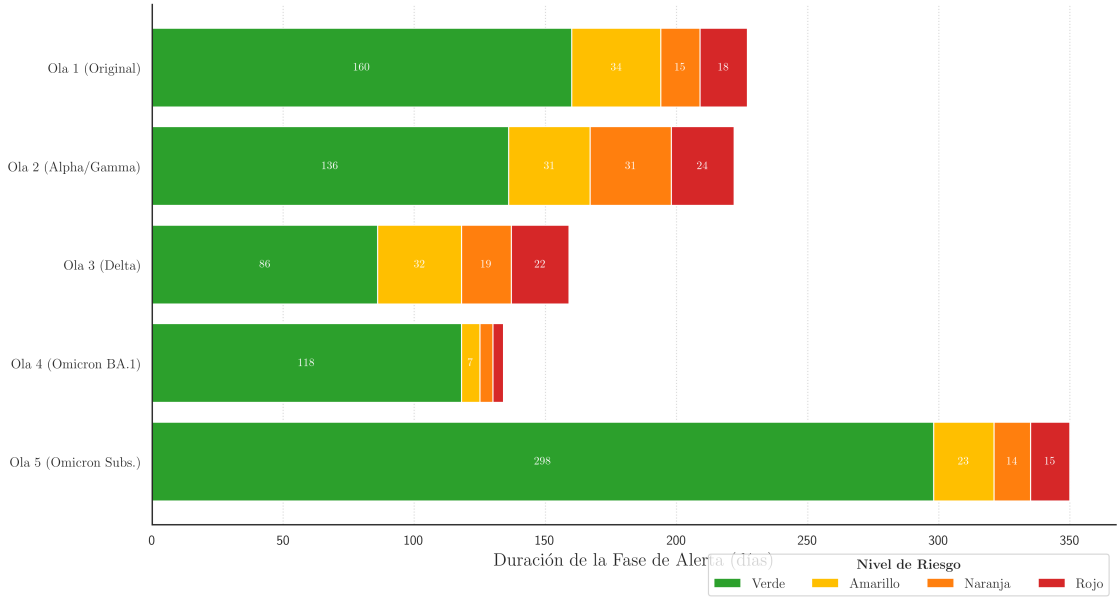
[29]:
```
# ================================================================================
# CELDA DEFINITIVA: FIGURA 6.6 - COMPARACIÓN DE LA DURACIÓN DE NIVELES DE ALERTA
# Versión final y sincronizada. Simula cada ola, cuenta los días en cada
# nivel de alerta y genera una gráfica comparativa de barras apiladas.
# ================================================================================
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from collections import Counter

# --- PARÁMETROS, FUNCIONES Y DATOS (Asumimos que ya existen en memoria) ---
# PARAMETROS_MAESTROS, EVENTOS_CLAVE, N_total, sir_model_ivp,
# olas_periodos, trayectorias_empiricas, calculate_alert_levels, ALERT_LEVELS

# --- CÁLCULO DE DURACIONES DE ALERTA PARA CADA OLA ---
print("Calculando duraciones de alerta para las 5 olas...")
resultados_alertas = []

for i in range(len(PARAMETROS_MAESTROS)):
    ola_idx = i + 1
    params = PARAMETROS_MAESTROS[ola_idx]
    fecha_inicio_ola = olas_periodos[i]['inicio']
    fecha_fin_ola = olas_periodos[i]['fin']

    # Simulación con inicio realista y duración de ola correcta
    gamma = 1.0 / params['D']
    s0_abs = params['S0_N'] * N_total
    i0_abs = trayectorias_empiricas[i]['I_real_abs'].iloc[0]
    r0_abs = N_total - s0_abs - i0_abs

    duracion_dias = (fecha_fin_ola - fecha_inicio_ola).days
    t_span = (0, duracion_dias)
    t_eval = np.arange(t_span[0], t_span[1])

    sol = solve_ivp(sir_model_ivp, t_span, [s0_abs, i0_abs, r0_abs],
                    t_eval=t_eval, args=(params['R0'], gamma, N_total),␣
 ↪method='RK45')

    # Calcular niveles de alerta diarios
    alert_levels, _, _ = calculate_alert_levels(sol.t, sol.y[0]/N_total, sol.
 ↪y[1]/N_total, params['R0'], gamma)

    # Contar los días en cada nivel
    counts = Counter(alert_levels)
    resultados_alertas.append({
        'nombre_ola': params['nombre'],
        1: counts.get(1, 0), # Días en Verde
        2: counts.get(2, 0), # Días en Amarillo
        3: counts.get(3, 0), # Días en Naranja
        4: counts.get(4, 0)  # Días en Rojo
    })
```

```
df_alertas = pd.DataFrame(resultados_alertas)
print("Cálculos completados.")

# --- CREACIÓN DE LA GRÁFICA DE BARRAS APILADAS ---
print("Generando Figura 6.6 de comparación...")
fig, ax = plt.subplots(figsize=(14, 8))

# Definir los niveles y colores para la gráfica
niveles = [1, 2, 3, 4]
colores = [ALERT_LEVELS[level][1] for level in niveles]
nombres_olas = df_alertas['nombre_ola']
bottoms = np.zeros(len(nombres_olas))

# Crear las barras apiladas
for i, nivel in enumerate(niveles):
    valores = df_alertas[nivel]
    bars = ax.barh(nombres_olas, valores, left=bottoms, color=colores[i],
                   edgecolor='white', label=ALERT_LEVELS[nivel][0])

    # Añadir etiquetas de texto dentro de las barras
    for bar, valor in zip(bars, valores):
        if valor > 5: # Solo etiquetar si el segmento es suficientemente grande
            ax.text(bar.get_x() + bar.get_width() / 2, bar.get_y() + bar.
 ↪get_height() / 2,
                    int(valor), ha='center', va='center', color='white',␣
 ↪weight='bold', fontsize=10)

    bottoms += valores

# --- Formato y Títulos ---
ax.set_title(r'\textbf{Figura 6.6: Comparación de la Duración de los Niveles de␣
 ↪Alerta por Ola}', fontsize=20, pad=15)
ax.set_xlabel('Duración de la Fase de Alerta (días)', fontsize=16)
ax.legend(title=r'\textbf{Nivel de Riesgo}', loc='lower right', ncol=4,␣
 ↪bbox_to_anchor=(1, -0.15))
ax.invert_yaxis() # Poner la Ola 1 arriba
ax.grid(True, which='major', axis='x', linestyle=':', alpha=0.7)
ax.spines[['top', 'right']].set_visible(False)

# --- Guardado ---
fig.tight_layout()
ruta_figura = 'Figuras Tesis/Figura_6_6_Comparacion_Alertas_FINAL.png'
fig.savefig(ruta_figura, dpi=300)
plt.show()

print(f"  Figura 6.6 (versión final) generada y guardada en '{ruta_figura}'.")
```

```
Calculando duraciones de alerta para las 5 olas…
Cálculos completados.
Generando Figura 6.6 de comparación…
```

Figura 6.6: Comparación de la Duración de los Niveles de Alerta por Ola




```
Figura 6.6 (versión final) generada y guardada en 'Figuras
Tesis/Figura_6_6_Comparacion_Alertas_FINAL.png'.
```

[ ]: